Highlights

GMM-Router: Density-Based Task-Agnostic Routing for Continual Code Generation

Tran An Khanh

- GMM-Router routes inputs to per-task LoRA experts via frozen-backbone GMM scores.
- A fixed orthonormal projection keeps the routing space stable as tasks accumulate.
- Highest score (51.62%) among task-agnostic methods on an 8-task CodeTask benchmark.
- Highest Pass@5 (47.75%) on a 9-language executable code generation benchmark.
- JSD-gated traceback routing recovers samples that fail every Round-1 candidate.

GMM-Router: Density-Based Task-Agnostic Routing for Continual Code Generation

Tran An Khanh[a,*]

[a]School of Information and Communication Technology, Hanoi University of Science and Technology, TODO, Hanoi, TODO, , Vietnam

ARTICLE INFO

Keywords:
continual learning
code generation
parameter-efficient fine-tuning
Gaussian mixture model
mixture of experts
catastrophic forgetting
large language models for code

ABSTRACT

Code language models deployed in practice must adapt continuously to new programming languages, task types, and coding standards, which gives rise to continual learning for code generation: a model is trained on a sequence of tasks but risks catastrophic forgetting of earlier ones. Existing approaches rely on parameter regularization, prompt tuning, or data replay, each of which struggles with structurally diverse task streams, offers limited expressive capacity, or raises data-retention concerns. This paper studies whether routing can avoid these limitations without task identity at inference. We propose GMM-Router, which trains a task-specific low-rank adaptation (LoRA) expert per task while a diagonal Gaussian Mixture Model (GMM), fitted by Expectation-Maximization over frozen, orthonormally projected backbone representations, provides task-agnostic scores that merge expert adapters via a temperature-scaled softmax. For executable code generation, an iterative refinement stage fits a second GMM on runtime error tracebacks and combines it with the instruction router through a Jensen–Shannon-divergence-gated rule to re-predict samples that fail on the first attempt. GMM-Router is evaluated on two complementary benchmarks: an eight-task extension of CodeTask-CL scored by text-overlap metrics, and a nine-language executable benchmark scored by unit-test execution. On the diverse CodeTask sequence, GMM-Router attains the highest average score (51.62%) among all task-agnostic baselines, with near-zero backward transfer (−1.01%). On the executable benchmark, it reaches 34.40% Pass@1 and the highest Pass@5 (47.75%) among all evaluated methods. These results indicate that combining architectural expert isolation with density-based probabilistic routing is an effective, practical strategy for task-agnostic continual code generation.

1. Introduction

Large language models for code have shown strong results on a wide range of software engineering tasks, including code generation from natural language, cross-language translation, code summarization, bug repair, and program synthesis (Wang, Wang, Joty and Hoi, 2021; Rozi‘ere, Gehring, Gloeckle, Sootla, Gat, Tan, Adi, Liu, Sauvestre, Remez et al., 2023; Hui, Yang, Cui, Yang, Liu, Zhang, Liu, Zhang, Yu, Lu et al., 2024; Lozhkov, Li, Ben Allal, Cassano, Lamy-Poirier, Tazi, Tang, Pykhtar, Liu, Wei et al., 2024; Lu, Guo, Ren, Huang, Svyatkovskiy, Blanco, Clement, Drain, Jiang, Tang, Li, Zhou, Shou, Zhou, Tufano, Gong, Zhou, Duan, Sundaresan, Deng, Fu and Liu, 2021; Chen, Tworek, Jun, Yuan, Pinto, Kaplan, Edwards, Burda, Joseph, Brockman et al., 2021; Austin, Odena, Nye, Bosma, Michalewski, Dohan, Jiang, Cai, Terry, Le et al., 2021). These results come mainly from large-scale offline training, in which data for all target tasks is collected in advance, combined into one corpus, and used to train the model over many iterations. This approach works well when the target task distribution is fixed, but it becomes harder to sustain as the number of tasks grows and deployment conditions change.

In long-term deployment, a code assistant cannot remain unchanged after its initial training. New programming languages emerge, existing frameworks change, and the range of capabilities expected of the model keeps growing. Retraining the model from scratch after each change is slow and inflexible, and in many business settings re-using old training data raises privacy concerns that limit approaches based on storing or replaying historical data. A more practical alternative lets the model absorb new tasks one at a time while continuing to handle previously learned tasks without access to old data. Standard supervised learning is not suited to this setting because it assumes the full training set is available at once and that the task distribution is stationary; neither assumption holds when a model is updated incrementally under storage or privacy constraints.

Continual learning studies exactly this problem: a model learns from a stream of tasks without forgetting what it has already learned (Parisi, Kemker, Part, Kanan and Wermter, 2019; De Lange, Aljundi, Masana, Parisot, Jia, Leonardis, Slabaugh and Tuytelaars, 2021). Updates that reduce the loss on the current task tend to overwrite knowledge from earlier tasks, a phenomenon known as catastrophic forgetting (McCloskey and Cohen, 1989; Goodfellow, Mirza, Xiao, Courville and Bengio, 2013). The code domain makes this

*Corresponding author
TODO@example.edu (T.A. Khanh)
ORCID(s):

problem particularly difficult because tasks vary widely in input format, output format, programming language, and evaluation criterion — a task sequence may move from English-to-Java generation to Java-to-C# translation, Ruby summarization, and multilingual executable program synthesis, a level of diversity comparable to the CodeTask-CL benchmark (Yadav, Sun, Ding, Li, Zhang, Tan, Ma, Bhatia, Nallapati, Ramanathan, Bansal and Xiang, 2023). Existing continual learning approaches have clear limitations in this setting. Regularization methods (Kirkpatrick, Pascanu, Rabinowitz, Veness, Desjardins, Rusu, Milan, Quan, Ramalho, Grabska-Barwinska et al., 2017; Zenke, Poole and Ganguli, 2017; Aljundi, Babiloni, Elhoseiny, Rohrbach and Tuytelaars, 2018) accumulate importance weights across tasks, but their penalty becomes unreliable on long and structurally diverse sequences. Prompt-based methods (Wang, Zhang, Ebrahimi, Sun, Zhang, Lee, Ren, Su, Perot, Dy and Pfister, 2022; Razdaibiedina, Mao, Hou, Khabsa, Lewis and Almahairi, 2023) keep the backbone frozen but have limited representational capacity for tasks as different as translation, summarization, and repair. Sequential low-rank adaptation methods (Wang, Chen, Ge, Xia, Bao, Zheng, Zhang, Gui and Huang, 2023b; Wang, Lyu, Sun and Jing, 2025; Qian, Gao, Ding, Zhou, Liang, Duan, Liu, Zhao and Tang, 2025) cause destructive interference within a shared low-rank subspace; orthogonality constraints reduce but do not eliminate this problem, and the number of available orthogonal directions shrinks as tasks accumulate.

A further limitation shared by most existing approaches is the assumption that task identity is available at inference time. Methods that drop this assumption typically rely on heuristics such as nearest-centroid assignment, which break down when task embeddings overlap — a common occurrence in code generation, where problems from different languages can produce nearly identical instruction embeddings. Current evaluation practice compounds these gaps: the most widely used continual learning benchmark for code (Yadav et al., 2023) covers only four tasks and is scored with text-similarity metrics such as BLEU (Papineni, Roukos, Ward and Zhu, 2002) and CodeBLEU (Ren, Guo, Lu, Zhou, Liu, Tang, Sundaresan, Zhou, Blanco and Ma, 2020), which cannot detect code that looks plausible but fails functionally, and short task sequences hide long-range forgetting effects. Few prior studies combine sequential learning with executable, multilingual, unit-test-based evaluation (Chen et al., 2021; Cassano, Gouwar, Nguyen, Nguyen, Phipps-Costin, Pinckney, Yee, Zi, Anderson, Feldman et al., 2023).

This paper addresses task-agnostic, parameter-efficient continual learning for sequential code generation. We adopt a routing-based approach that combines per-task expert isolation with a learned probabilistic model for task identification at inference time, avoiding data replay, parameter interference, and the requirement for explicit task labels. The contributions of this paper are as follows.

leftmirgin=* **A two-track evaluation platform for continual code generation.** We extend CodeTask-CL (Yadav et al., 2023) with four additional tasks covering new programming languages and task types, producing a longer, more diverse eight-task sequence scored by text-overlap metrics. We additionally build a nine-language executable benchmark from OSS-Instruct, McEval-Instruct, McEval, MBPP, and MultiPL-E (Wei, Wang, Liu, Ding and Zhang, 2023; Chai, Liu, Yang, Yin, Jin, Liu, Sun, Zhang, Ren, Guo, Wang, Wang, Wu, Wang, Li, Yang, Duan and Li, 2025; Austin et al., 2021; Cassano et al., 2023), in which generated programs are evaluated by running them against unit tests.

leftmiirgiin=* **A deduplication pipeline for the executable benchmark** that combines exact matching via normalized SHA-256 hashing with a near-duplicate detection stage based on shingling, MinHash, Locality-Sensitive Hashing, and exact Jaccard scoring (Jaccard, 1901; Broder, 1997; Indyk and Motwani, 1998), reducing train–test leakage.

leftmiiirgiiin=* **A systematic empirical comparison** of regularization-based, prompt-based, low-rank-adaptation-based, and online continual learning baselines on sequential programming tasks under a consistent training and evaluation protocol.

leftmivrgivn=* **GMM-Router**, a routing-based continual learning framework that trains a task-specific LoRA expert per task on a frozen backbone, fits a diagonal Gaussian Mixture Model by Expectation-Maximization on routing embeddings extracted through a fixed orthonormal projection, and merges expert adapters at the weight level using temperature-scaled softmax routing weights. For executable code generation, the framework further incorporates a traceback-based router that uses runtime failure signals to guide a second inference round.

We evaluate GMM-Router against seven continual learning baselines and a task-identity upper bound

on both benchmark tracks. On the diverse CodeTask benchmark, GMM-Router achieves the highest average score (51.62%) among all task-agnostic methods, approaching the upper bound (54.15%) with near-zero forgetting. On the executable benchmark, it achieves 34.40% Pass@1 and the highest Pass@5 (47.75%) among all methods evaluated.

The remainder of this paper is organized as follows. Section 2 presents the motivation and the conceptual background needed to follow the proposed method. Section 3 describes the data collection, formatting, and deduplication pipeline underlying the two benchmark tracks. Section 4 presents the GMM-Router methodology. Section 5 defines the research questions and the experimental configuration. Section 6 reports and discusses the experimental results. Section 7 situates GMM-Router within the continual learning and mixture-of-experts literature. Section 8 concludes the paper and outlines directions for future work.

2. Motivation and Background

2.1. Motivation

A code assistant deployed for an extended period is rarely trained once and left unchanged. New programming languages and frameworks appear, task requirements shift, and the range of requests a model must handle keeps expanding. Two constraints make this update problem hard in practice. First, full retraining on the union of all data seen so far is computationally expensive and grows more so as the task stream lengthens. Second, data from earlier tasks — proprietary code, private repositories, or controlled benchmark collections — may become unavailable or unusable for retraining because of storage or privacy constraints. A practical solution must therefore let the model absorb each new task using only that task's data, while retaining acceptable performance on every task learned previously, and it must do so without knowing in advance which task an incoming request belongs to.

This paper treats sequential code generation as a case study for this problem because the tasks that a code assistant is asked to perform are structurally heterogeneous: some map natural language to code, some map code to code, and some map code to natural language, across many programming languages and syntactic conventions. This heterogeneity means that a continual learning method effective for narrow, homogeneous task streams — as is common in image classification benchmarks — does not automatically transfer to code. The remainder of this section introduces the concepts needed to state the problem precisely and to follow the GMM-Router methodology in Section 4: code language models as the pretrained backbones used throughout this study (Section 2.2), the continual learning problem formulation (Section 2.3), parameter-efficient fine-tuning as the adaptation mechanism (Section 2.4), and Gaussian Mixture Models as the probabilistic tool used for routing (Section 2.5). Detailed comparison with prior continual learning and routing methods is deferred to Section 7.

2.2. Code language models

Large language models pretrained on source code follow one of two broad architectural families. Encoder-decoder models use a bidirectional encoder together with a causal decoder and are well suited to tasks such as translation and summarization, as in CodeT5 (Wang et al., 2021) and CodeT5+ (Wang, Le, Gotmare, Bui, Li and Hoi, 2023c). Decoder-only models use a single stack of causal self-attention layers that processes both the input and the generated output, which is more natural for open-ended generation and completion; this family includes Code Llama (Rozi‘ere et al., 2023), StarCoder2 (Lozhkov et al., 2024), and Qwen2.5-Coder (Hui et al., 2024). All of these families are built on the Transformer architecture (Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser and Polosukhin, 2017). Qwen2.5-Coder, a decoder-only model built for code generation, completion, reasoning, and repair, serves as the backbone throughout this paper (Section 5.3). Model architecture matters for continual learning because a model updated repeatedly over time must preserve both its representations and its generation abilities across a growing task sequence; this consideration motivates keeping the backbone frozen and confining task-specific updates to a small set of parameters, as described next.

2.3. Continual learning

Continual learning — also called lifelong or sequential learning — is a setting in which a model is trained on a stream of tasks that arrive one at a time (Parisi et al., 2019; De Lange et al., 2021). Let $\mathcal{T}_1, \ldots, \mathcal{T}_K$ denote a sequence of tasks, each associated with a training dataset

$$\mathcal{D}_k = \left\{ (x_i^{(k)}, y_i^{(k)}) \right\}_{i=1}^{N_k}, \tag{1}$$

where $x_i^{(k)}$ is an input prompt and $y_i^{(k)}$ is the target output sequence. The naive sequential update minimizes the current task loss starting from the previous parameters θ_{k-1}: $\theta_k = \arg\min_\theta \mathbb{E}_{(x,y)\sim\mathcal{D}_k}[\ell(f_\theta(x), y)]$. This update is simple but typically causes catastrophic forgetting (McCloskey and Cohen, 1989; Kirkpatrick et al., 2017): updates that reduce loss on the current task overwrite parameters that encoded earlier-task knowledge. In code generation, a model may lose Java-to-C# translation ability after fine-tuning on Ruby summarization, or degrade on Python generation after adapting to Rust. An effective continual learner must balance stability (retaining earlier tasks) against plasticity (acquiring new ones) (De Lange et al., 2021); this trade-off is pronounced in code, where a task

sequence may mix closely related tasks that share useful representations with structurally different tasks that require separate generation behavior. Section 5.3.3 defines the Forgetting Measure, Backward Transfer, and Forward Transfer metrics used throughout this paper to quantify this trade-off.

This paper studies the setting in which task identity is not provided at inference time: the model must infer, from the input alone, which adaptation path to use. This requirement motivates the routing formulation developed in Section 4.

2.4. Parameter-efficient fine-tuning

Parameter-efficient fine-tuning (PEFT) methods introduce or modify a small number of trainable parameters while keeping the pretrained backbone frozen, reducing per-task memory and compute cost and making it practical to maintain separate adaptation modules across a growing task sequence. Prompt tuning (Lester, Al-Rfou and Constant, 2021) prepends a set of trainable continuous embeddings to the input sequence while leaving backbone parameters untouched. Adapter modules (Houlsby, Giurgiu, Jastrzebski, Morrone, De Laroussilhe, Gesmundo, Attariyan and Gelly, 2019) insert a bottleneck transformation $h' = h + W_{\text{up}}\,\sigma(W_{\text{down}}\,h)$ inside each transformer layer, where $W_{\text{down}} \in \mathbb{R}^{r\times d}$ projects to a bottleneck of size $r \ll d$.

LoRA (Hu, Shen, Wallis, Allen-Zhu, Li, Wang, Wang and Chen, 2022), the PEFT method used throughout this paper, factorizes a weight update into two low-rank matrices:

$$W = W_0 + \Delta W, \qquad \Delta W = \frac{\alpha}{r} BA, \tag{2}$$

where $W_0 \in \mathbb{R}^{d\times k}$ is the frozen pretrained weight, $A \in \mathbb{R}^{r\times k}$ and $B \in \mathbb{R}^{d\times r}$ are trainable low-rank factors initialized so that $\Delta W = 0$ at the start of training, $r \ll \min(d, k)$, and α is a fixed scaling hyperparameter. LoRA offers a strong balance between efficiency and expressiveness and is easy to insert into attention projection layers. For continual learning, PEFT alone reduces — but does not eliminate — interference: a single shared LoRA or prompt can still be overwritten by later tasks. Section 4 uses per-task LoRA experts to remove this source of interference by construction, and combines them with the density-based routing mechanism introduced next.

2.5. Gaussian Mixture Models

A Gaussian Mixture Model (GMM) represents a data distribution as a weighted combination of Gaussian components (Bishop, 2006). Figure 1 illustrates a two-component example: the two component densities (dashed) sum to a bimodal mixture density (solid) that a single Gaussian could not represent. For high-dimensional representations it is common to restrict each component's covariance to a diagonal matrix. A

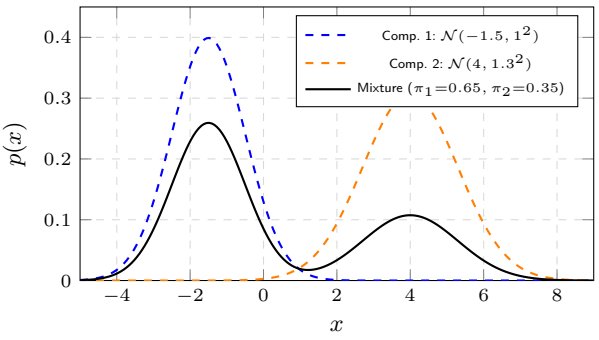


Figure 1: A two-component Gaussian Mixture Model. Each dashed curve is an individual component; the solid curve is their weighted sum. The mixture density is bimodal, whereas a single Gaussian would be forced to place mass between the two clusters.

diagonal GMM with M components over a representation $\mathbf{z}$ is

$$p(\mathbf{z}) = \sum_{m=1}^{M} \pi_m \,\mathcal{N}\big(\mathbf{z};\, \boldsymbol{\mu}_m,\, \operatorname{diag}(\boldsymbol{\sigma}_m^2)\big)\,, \tag{3}$$

where $\pi_m \geq 0$, $\sum_{m=1}^{M} \pi_m = 1$ are mixture weights, $\boldsymbol{\mu}_m \in \mathbb{R}^p$ are component means, and $\boldsymbol{\sigma}_m^2 \in \mathbb{R}^p_{>0}$ collects per-dimension variances. The log-density of a point $\mathbf{z}$ under component m is

$$\log \mathcal{N}\big(\mathbf{z}; \boldsymbol{\mu}_m, \boldsymbol{\sigma}_m^2\big) = -\frac{1}{2}\Bigg[p\ln(2\pi) + \sum_{j=1}^{p} \ln \sigma_{mj}^2 + \sum_{j=1}^{p} \frac{(z_j - \mu_{mj})^2}{\sigma_{mj}^2}\Bigg], \tag{4}$$

and the marginal log-density is computed with the log-sum-exp trick for numerical stability:

$$\log p(\mathbf{z}) = \operatorname{logsumexp}_m \big[\log \pi_m + \log \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}_m, \boldsymbol{\sigma}_m^2)\big]\,. \tag{5}$$

Because component assignment is a latent variable, GMM parameters are fitted by maximum likelihood using Expectation-Maximization (EM) (Dempster, Laird and Rubin, 1977). The E-step computes the responsibility of component m for observation $\mathbf{z}_i$,

$$r_{im} = \frac{\pi_m \,\mathcal{N}(\mathbf{z}_i; \boldsymbol{\mu}_m, \boldsymbol{\sigma}_m^2)}{\sum_{m'} \pi_{m'} \,\mathcal{N}(\mathbf{z}_i; \boldsymbol{\mu}_{m'}, \boldsymbol{\sigma}_{m'}^2)}, \tag{6}$$

and the M-step updates parameters using effective counts $N_m = \sum_i r_{im}$:

$$\pi_m = \frac{N_m}{N}, \qquad \boldsymbol{\mu}_m = \frac{1}{N_m} \sum_{i=1}^{N} r_{im} \mathbf{z}_i, \tag{7}$$

$$\sigma_{mj}^2 = \max\left(\frac{\sum_{i=1}^{N} r_{im}(z_{ij} - \mu_{mj})^2}{N_m}, \sigma_{\min}^2\right), \tag{8}$$

Algorithm 1: Expectation-Maximization for a Diagonal GMM

Input: Observations $\{\mathbf{z}_i\}_{i=1}^{N}$; components M; tolerance δ; max steps $T_{\max}$
Output: Fitted $\{\pi_m, \boldsymbol{\mu}_m, \boldsymbol{\sigma}_m^2\}_{m=1}^{M}$
Initialize $\pi_m \leftarrow 1/M$; sample $\boldsymbol{\mu}_m$ uniformly from $\{\mathbf{z}_i\}$; set $\boldsymbol{\sigma}_m^2$ to the global per-dimension variance;
$\mathcal{L}^{(0)} \leftarrow -\infty$;
for $t = 1, \ldots, T_{\max}$ **do**
  E-step: compute r_{im} for all i, m via Eq. (6);
  M-step: update $\pi_m, \boldsymbol{\mu}_m, \boldsymbol{\sigma}_m^2$ for all m via Eq. (8);
  $\mathcal{L}^{(t)} \leftarrow (1/N) \sum_{i=1}^{N} \log p(\mathbf{z}_i)$ via Eq. (5);
  if $|\mathcal{L}^{(t)} - \mathcal{L}^{(t-1)}| < \delta$ **then**
    break
return $\{\pi_m, \boldsymbol{\mu}_m, \boldsymbol{\sigma}_m^2\}_{m=1}^{M}$;

where $\sigma_{\min}^2 > 0$ is a variance floor that prevents component collapse. Algorithm 1 summarizes the procedure, alternating the E-step (Eq. (6)) and the M-step (Eq. (8)) until the average log-likelihood converges.

GMM-based routing decides which adaptation module to apply to a given input by estimating how likely its feature representation is under each task's fitted density; this connects to the mixture-of-experts literature, in which a gating function assigns inputs to specialized expert networks (Jacobs, Jordan, Nowlan and Hinton, 1991; Jordan and Jacobs, 1994; Shazeer, Mirhoseini, Maziarz, Davis, Le, Hinton and Dean, 2017; Fedus, Zoph and Shazeer, 2022), reviewed further in Section 7. In the code domain, a single task often contains diverse subtypes — algorithmic problems, string processing, data-structure manipulation — that a single Gaussian cannot represent well; a mixture captures several subregions within the same task, and evaluating each task's fitted GMM on a frozen, task-agnostic representation provides a routing score without requiring task labels at inference time, motivating the routing mechanism developed in Section 4.

3. Data Collection and Preprocessing

Two complementary benchmark tracks support the evaluation in this paper: a non-executable track scored by text-overlap metrics, and an executable track scored by unit-test execution. Figure 2 summarizes the overall data pipeline, which proceeds through five stages: raw datasets are collected from multiple sources, schema-normalized to a common format, formatted using instruction templates, deduplicated to remove identical or near-identical examples, and partitioned into train, calibration, and test splits.

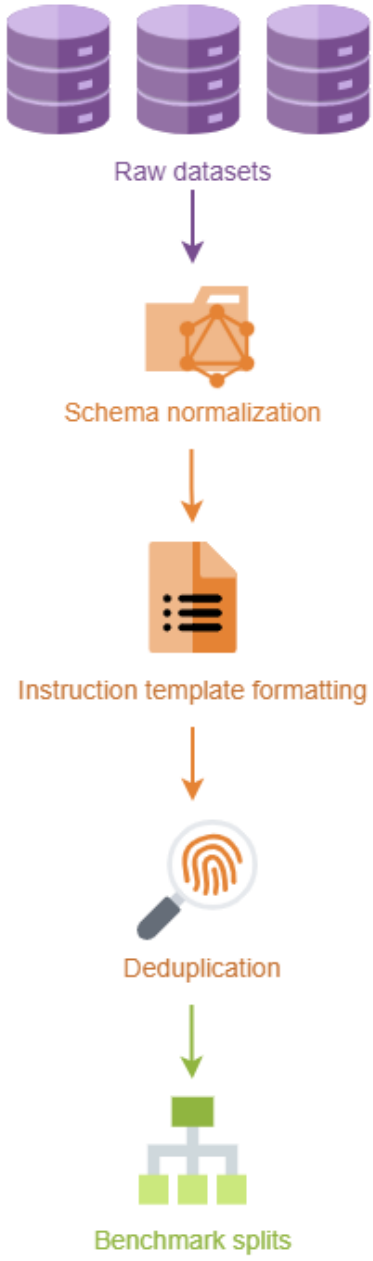


Figure 2: Data collection and preprocessing pipeline. Five sequential stages transform raw datasets from multiple diverse sources into clean, unified benchmark splits for both continual learning evaluation tracks.

3.1. The CodeTask benchmark

The CodeTask benchmark is the non-executable track used in this paper. It extends CodeTask-CL (Yadav et al., 2023), which combines generation, translation, summarization, and refinement tasks, with four additional tasks to create a longer and more diverse continual learning sequence. The original setting contains four tasks: English-to-Java generation using CONCODE (Iyer, Konstas, Cheung and Zettlemoyer,

Table 1
The CodeTask benchmark used in the non-executable continual learning track.

Task	Dataset	In	Out	Train	Val.	Test
Generation	CONCODE	English	Java	100,000	2,000	2,000
Translation	CodeXGLUE CodeTrans	Java	C#	10,300	500	1,000
Summarization	CodeSearchNet	Ruby	English	25,000	1,400	1,200
Refinement	BFP	Java	Java	46,600	5,800	5,800
Generation	KodCode	English	Python	100,000	5,000	5,000
Refinement	RunBugRun	Ruby	Ruby	10,000	972	1,000
Summarization	the-vault-function	C#	English	100,000	5,000	5,000
Translation	CoST	C++	C#	12,000	272	410

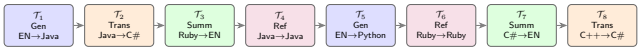


Figure 3: The eight-task CodeTask continual learning sequence. Training data from earlier tasks are unavailable once the model advances past each task boundary. Node color indicates task type: blue for generation, orange for translation, green for summarization, purple for refinement.

2018), Java-to-C# translation using CodeXGLUE CodeTrans (Lu et al., 2021), Ruby-to-English summarization using CodeSearchNet (Husain, Wu, Gazit, Allamanis and Brockschmidt, 2019), and Java-to-Java refinement using BFP (Tufano, Watson, Bavota, Di Penta, White and Poshyvanyk, 2019). We add Python generation using KodCode (Xu, Liu, Yin, Zhou and Poovendran, 2025), Ruby refinement using RunBugRun (Prenner and Robbes, 2023), C#-to-English summarization using the-vault-function (Nguyen, Le, Dau, Nguyen, Nghiem, Guo and Bui, 2023), and C++-to-C# translation using CoST (Zhu, Suresh, Korotkova, Aghajanyan, O'Gorman, Iyyer and Reddy, 2022). Table 1 lists the eight tasks with their dataset sources and split sizes; Figure 3 shows the resulting task order.

The additional tasks widen the range of languages and task types and introduce stronger distribution shifts: some tasks map natural language to code, some map code to code, and some map code to natural language. This mix makes the sequence more representative of real code-assistant workloads and better suited to studying when knowledge should transfer between tasks and when it should remain separated; restricting the sequence to one type of generation problem would risk conflating narrow adaptation with genuine continual learning ability.

Each dataset is cast into a unified instruction-style sequence-to-sequence format, since the source datasets use different schemas (natural-language intent and code snippet; buggy and repaired code; source code and docstring). To avoid the model relying on a single memorized prompt format, each example is wrapped with a template drawn from a pool of $N = 20$ training templates per task type; at test time the pool expands to $M = 30$ templates, so the test-time instructions include both familiar and unseen phrasing.

Table 2
Statistics of the executable benchmark subsets.

Split	Py	C++	C#	Java	Rust	Swift	PHP	TS	Shell	Total
Train	5,699	5,697	5,449	5,565	5,692	5,496	5,576	5,698	5,726	50,598
Calib.	426	397	386	386	354	396	397	390	382	3,514
Test	50	50	50	53	53	50	50	50	50	456

3.2. The executable benchmark and deduplication pipeline

The executable benchmark evaluates multilingual code generation under execution-based grading. Training data are drawn from OSS-Instruct (Wei et al., 2023) and McEval-Instruct (Chai et al., 2025); test data are drawn from McEval (Chai et al., 2025). All subsets cover the same nine programming languages: Python, C++, C#, Java, Rust, Swift, PHP, TypeScript, and Shell. Unlike the CodeTask benchmark, each test example provides a problem description together with unit tests rather than a single reference output, and success is defined by whether the generated program passes the tests rather than by textual similarity to a reference solution, following the execution-based evaluation methodology of Chen et al. (2021).

Table 2 summarizes the train, calibration, and test splits. OSS-Instruct is the largest training source (41,613 examples); McEval-Instruct adds 8,985 more balanced examples; McEval serves as the held-out test set. The calibration split — used for candidate generation and for fitting the traceback router described in Section 4 — is assembled from the sanitized MBPP test split for Python (Austin et al., 2021) and from the `mbpp-ext` configuration of MultiPL-E for the remaining eight languages (Cassano et al., 2023); MultiPL-E does not provide reference solutions for these languages, so only pass/fail execution feedback is available for them. All calibration and test data are standardized to match the McEval instruction and test format.

The benchmark also brings multilingual adaptation into the continual learning setting: the same code-generation task repeats across nine languages that differ in syntax, standard libraries, and idiomatic patterns, so a continual learner may experience both positive transfer and interference across languages that the CodeTask benchmark, with its diversity of task types, does not directly test. Compared with CodeTask, the executable benchmark is narrower in task type but stronger in functional evaluation, isolating a single question: can the model keep learning to generate working programs across languages over time?

Because the executable benchmark is graded by execution, an undetected overlap between training and test examples would inflate reported scores by rewarding memorization rather than generalization, an effect that matters even more for a continual learner, whose apparent retention could otherwise reflect data leakage rather than genuine forward or backward transfer. The construction of the executable benchmark therefore

includes a two-stage deduplication pipeline, illustrated in Figure 4.

Exact deduplication. Instruction text is whitespace-normalized and hashed with SHA-256 (National Institute of Standards and Technology, 2015); examples sharing a hash are grouped into a cluster and reduced to one canonical representative. This stage removes verbatim copies but not near-identical problems that differ only by renamed identifiers or reworded descriptions.

Near deduplication. A stronger normalization is applied first — Unicode normalization, prose/code separation, placeholder abstraction, identifier generalization to generic tokens, and import/punctuation normalization — after which each example is represented as a set of token n-gram shingles. Comparing all shingle-set pairs directly is $O(N^2)$ and infeasible at scale, so MinHash is used to approximate Jaccard similarity efficiently: given m independent hash functions, the signature of a shingle set S is $\mathrm{MH}(S) = (\min_{s\in S} h_1(s), \ldots, \min_{s\in S} h_m(s))$, with the property $\Pr[\min h(S_A) = \min h(S_B)] = J(S_A, S_B)$ (Broder, 1997). Locality-Sensitive Hashing (LSH) then restricts pairwise comparison to signatures sharing a band (Indyk and Motwani, 1998), and each surviving candidate pair is scored with the exact Jaccard similarity $J(S_A, S_B) = |S_A \cap S_B|/|S_A \cup S_B|$ (Jaccard, 1901); a pair with $J \geq \theta$ is recorded as a near duplicate.

Exact-duplicate clusters are filtered and near-duplicate pairs that cross the train–test boundary are removed; this train–test filtering step is the primary safeguard against the model solving a test problem because it has seen an almost-identical version during training rather than because it generalized. The final stage standardizes all processed outputs into a common schema across splits, with shared fields index, language, instruction, solution, and test, so deduplication is integrated into the main benchmark construction pipeline rather than a separate post-hoc cleanup step.

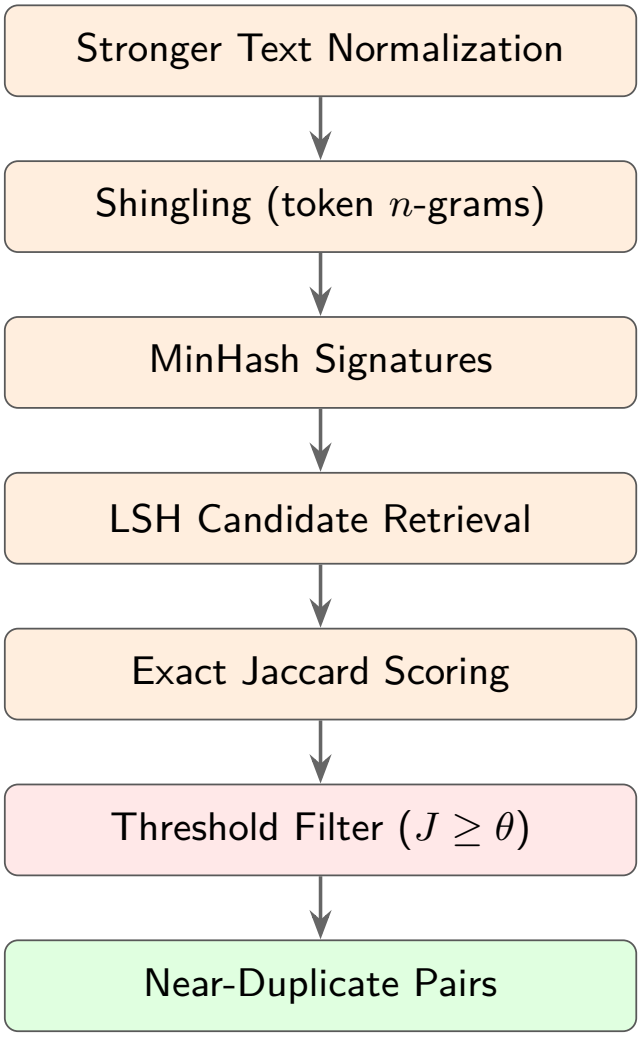


Figure 4: Near-deduplication pipeline. Normalization and shingling produce set representations; MinHash enables approximate similarity estimation; LSH restricts the search space to candidate pairs; exact Jaccard scoring provides the final decision.

4. Methodology

This section presents GMM-Router, a framework for parameter-efficient continual code generation that routes each query to task-specific experts without retraining earlier adapters or retaining raw training data. GMM-Router has four components: task-specific LoRA expert training (Section 4.1); routing feature extraction through a frozen backbone and a fixed orthonormal projection (Section 4.2); a continual GMM instruction router that supports hard and soft routing (Sections 4.3 and 4.4); and, for the executable benchmark, an iterative refinement pipeline built around a second, traceback-based router (Section 4.5).

At a high level, GMM-Router processes the task sequence as follows. For each new task, a dedicated LoRA adapter and a GMM instruction router are trained in parallel (Sections 4.1–4.3). At inference, the instruction router scores the query against every trained task and produces a routing distribution over adapters, which is used to select the highest-weighted adapter for hard routing or to merge all adapters for soft routing (Section 4.4); this constitutes Round-1 inference and is the only inference mechanism used on the CodeTask benchmark. On the executable benchmark, adapters are additionally evaluated on a held-out calibration split (Section 5.3) whose failed predictions are used to refine each adapter on repair pairs and to fit a traceback GMM on error messages (Section 4.5); for any test sample whose five Round-1 candidates all fail execution, a second inference round combines the instruction and traceback routing signals through a Jensen–Shannon-divergence (JSD) gated rule to produce one additional prediction (Section 4.5).

4.1. Training task-specific experts

For each task $\mathcal{T}_k$ with training dataset $\mathcal{D}_k$ (Eq. (1)), GMM-Router trains a task-specific LoRA adapter (Hu et al., 2022) on top of a shared frozen backbone, following the LoRA formulation in Section 2.4. Each adapter is a low-rank update applied to the query and value projections in every transformer layer. Only the adapter parameters $\Theta_k = \{B_k^{(\ell)}, A_k^{(\ell)}\}_\ell$ are updated; the backbone and all previously trained adapters remain frozen throughout training, so a new task cannot interfere with previously learned ones. The training objective is the negative log-likelihood of the target sequence,

$$\mathcal{L}_k(\Theta_k) = -\sum_{(x,y)\in\mathcal{D}_k}\sum_{t=1}^{|y|}\log p_k(y_t \mid x, y_{<t}), \qquad (9)$$

where $p_k(y_t \mid x, y_{<t})$ is the output distribution of expert k at decoding step t. The backbone is split at layer ℓ^*: layers 0 through ℓ^*-1 produce the frozen features used for routing (Section 4.2), while LoRA adapters modify layers ℓ^* through L. A larger ℓ^* dedicates more layers to routing features but leaves fewer layers for task-specific adaptation.

4.2. Routing feature extraction

Stable routing requires a fixed representation space: if the feature map changed as new tasks arrived, GMMs fitted on earlier tasks would become misaligned with those fitted later. GMM-Router achieves this by extracting features from the frozen part of the backbone — with all LoRA adapters disabled, so the routing space is independent of any per-task adapter weights — and projecting them with a random matrix drawn once and fixed for the entire continual training run.

For a tokenized input of length T, the backbone produces hidden states $\mathbf{H}^{(\ell)} \in \mathbb{R}^{T\times d}$ at each layer ℓ; the routing representation is taken at layer ℓ^*, $\mathbf{H} = \mathbf{H}^{(\ell^*)}$. Using an intermediate layer yields representations that are less task-specific and more stable as tasks accumulate. The sequence-level representation is the mean of the token hidden states,

$$\tilde{\mathbf{h}} = \frac{1}{T}\sum_{t=1}^{T}\mathbf{h}_t \in \mathbb{R}^d, \qquad (10)$$

and parameter-free layer normalization removes scale differences,

$$\mathbf{h} = \mathrm{LN}(\tilde{\mathbf{h}}), \qquad \mathrm{LN}(\mathbf{v}) = \frac{\mathbf{v}-\mu_{\mathbf{v}}}{\sigma_{\mathbf{v}}+\varepsilon}, \qquad (11)$$

where $\mu_{\mathbf{v}}$ and $\sigma_{\mathbf{v}}$ are the mean and standard deviation of the elements of $\mathbf{v}$. A fixed projection matrix $\mathbf{P} \in \mathbb{R}^{p\times d}$ then reduces the dimension to $p \ll d$: a random Gaussian matrix is drawn with a fixed seed, its thin QR factorization gives $\mathbf{P}$, so that $\mathbf{P}\mathbf{P}^\top = \mathbf{I}_p$. The routing embedding for input x is

$$\mathbf{z} = \mathbf{P}\,\mathbf{h} \in \mathbb{R}^p. \qquad (12)$$

Because $\mathbf{P}$ never changes, the routing embedding (Eq. (12)) lies in a fixed p-dimensional space for every task, so GMM densities fitted at any point in the sequence remain directly comparable to those fitted later.

4.3. Continual GMM instruction router

Each task's instruction distribution is modeled as a diagonal GMM with M components (Eq. (3)), fitted by EM (Algorithm 1) on the routing embeddings $\{\mathbf{z}_i^{(k)}\}_{i=1}^N$ of task k. When task k arrives, its GMM is fitted and appended to the router registry; no parameters belonging to earlier tasks are modified.

MAP scoring with a uniform class prior. At inference, the router scores each registered task k using the log-posterior under a uniform prior over the K trained tasks,

$$s_k(\mathbf{z}) = \log p_k(\mathbf{z}) + \log\frac{1}{K}. \qquad (13)$$

Since $\log(1/K)$ is constant across tasks, the argmax decision is equivalent to selecting the highest log-likelihood; the prior term keeps scores on a consistent scale as K grows.

Hard routing. Given a query embedding $\mathbf{z}$, the hard router selects the task with the highest score,

$$\hat{k} = \operatorname*{argmax}_{k\in\{1,\dots,K\}} s_k(\mathbf{z}), \qquad (14)$$

and generation proceeds using only expert $\hat{k}$'s adapter. This is the default inference strategy for the CodeTask benchmark.

4.4. Soft routing and adapter merging

Rather than committing to a single expert, soft routing converts the log-posterior scores into a probability distribution over all K tasks via a temperature-scaled softmax,

$$\alpha_k(x) = \frac{\exp(s_k(\mathbf{z})/\tau)}{\sum_{j=1}^{K}\exp(s_j(\mathbf{z})/\tau)}, \qquad \boldsymbol{\alpha}(x) \in \Delta^{K-1}, \quad (15)$$

where $\tau > 0$ is the routing temperature; as $\tau \to 0^+$, $\boldsymbol{\alpha}(x) \to \mathbf{e}_{\hat{k}}$ recovers hard routing, and as $\tau \to \infty$, $\boldsymbol{\alpha}(x) \to \mathbf{1}/K$ becomes uniform. The K LoRA adapters are then merged at the weight level,

$$\Delta W_{\text{merged}} = \sum_{k=1}^{K}\alpha_k(x)\,B_k A_k, \qquad (16)$$

yielding a single merged adapter applied to the frozen backbone for generation, at the cost of only one forward pass per decoding step regardless of K. Weight-level merging (Eq. (16)) approximates the mixture-of-experts output distribution $p_{\text{mix}}(y_t \mid x, y_{<t}) = \sum_k \alpha_k(x)\,p_k(y_t \mid x, y_{<t})$, which would otherwise require K independent forward passes; because $\alpha_k(x)$ depends

Algorithm 2: Soft-Routing Expert-Mixture Generation

Input: Query x; log-posterior scores $\{s_k(\mathbf{z})\}$; adapters $\{(B_k, A_k)\}_{k=1}^{K}$; temperature τ
Output: Generated output $\hat{y}$
Extract routing embedding $\mathbf{z}$ with LoRA adapters disabled;
$\alpha_k \leftarrow \exp(s_k(\mathbf{z})/\tau) / \sum_j \exp(s_j(\mathbf{z})/\tau)$, $\forall k$;
Merge adapters: $\Delta W \leftarrow \sum_k \alpha_k B_k A_k$;
Generate $\hat{y}$ with the merged adapter;
return $\hat{y}$;

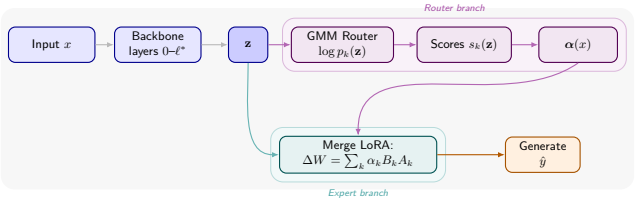


Figure 5: Round-1 inference pipeline of GMM-Router. The frozen backbone maps input x to routing embedding $\mathbf{z}$ using layers 0–ℓ^*. The router branch (purple) evaluates the K GMM log-densities, adds the uniform log-prior, and converts scores to routing weights $\boldsymbol{\alpha}(x)$. The expert branch (teal) merges the K LoRA adapters as $\Delta W = \sum_k \alpha_k B_k A_k$ and generates output $\hat{y}$.

only on x, the routing weights are computed once per query and held fixed throughout autoregressive decoding. Algorithm 2 summarizes soft-routing generation, and Figure 5 illustrates the resulting Round-1 inference pipeline.

4.5. Iterative refinement for executable code generation

For executable code generation, GMM-Router supports a second refinement round for samples where all Round-1 predictions fail execution. A separate GMM traceback router is trained on runtime error messages rather than instruction prompts, providing a complementary routing signal, and each per-task adapter is optionally refined on repair pairs; both procedures use the calibration predictions described in Section 5.3.

Traceback router. The traceback router is a diagonal GMM (Section 2.5) fitted on error-message embeddings extracted from failed calibration predictions. For each failed candidate with a non-empty error string, the error text is collected and deduplicated; the resulting strings are encoded with the same frozen backbone used for instruction routing and projected into a separate routing space with its own fixed projection matrix, and a GMM is fitted per task using the EM procedure of Section 2.5.

Adapter refinement. In parallel with traceback router training, each per-task LoRA adapter can be further fine-tuned on repair pairs derived from the evaluated calibration data. Each pair consists of the original instruction, buggy code, and runtime traceback as input, paired with a ground-truth or passing candidate as target: for each calibration sample, the first failed candidate with a non-empty error string supplies the (buggy code, traceback) pair, and the repair target is the first passing candidate, or the ground-truth solution if none of the five candidates passed; samples with no failed candidate or no repair target are skipped.

Round-2 inference and JSD-gated score combination. Round-2 inference applies only to samples where every Round-1 candidate failed. Let $\mathbf{s}_{\text{in}} \in \mathbb{R}^K$ be the instruction router's Round-1 log-posterior scores and $\mathbf{s}_{\text{tb}} \in \mathbb{R}^K$ the traceback router's scores on the corresponding error message; each is converted to a distribution, $\boldsymbol{\alpha}_{\text{in}} = \text{softmax}(\mathbf{s}_{\text{in}}/\tau)$ and $\boldsymbol{\alpha}_{\text{tb}} = \text{softmax}(\mathbf{s}_{\text{tb}})$. The two routers use fundamentally different signals — natural-language problem descriptions versus runtime error messages — so their agreement is informative about routing confidence: when both assign high probability to the same expert, that consensus is strong evidence the expert is correct; when they disagree, neither signal is fully trustworthy in isolation, and committing to either alone risks compounding the original routing error.

This motivates gating the combination by the degree of conflict between the two distributions. Let $\mathbf{m} = \frac{1}{2}(\boldsymbol{\alpha}_{\text{in}} + \boldsymbol{\alpha}_{\text{tb}})$. The Kullback–Leibler divergence from p to q is $\text{KL}(p\|q) = \sum_k p_k \log(p_k/q_k)$ (Kullback and Leibler, 1951), and the Jensen–Shannon divergence (Lin, 1991) is its symmetrized, bounded variant,

$$\text{JSD}(\boldsymbol{\alpha}_{\text{in}}\|\boldsymbol{\alpha}_{\text{tb}}) = \tfrac{1}{2}\text{KL}(\boldsymbol{\alpha}_{\text{in}}\|\mathbf{m}) + \tfrac{1}{2}\text{KL}(\boldsymbol{\alpha}_{\text{tb}}\|\mathbf{m}). \quad (17)$$

The combined routing weight is

$$\boldsymbol{\alpha} = (1 - \text{JSD})\,\text{softmax}(\mathbf{s}_{\text{in}} + \mathbf{s}_{\text{tb}}) + \text{JSD} \cdot \frac{\mathbf{1}}{K}. \quad (18)$$

When the two routers agree (low JSD), the combined log-posterior $\mathbf{s}_{\text{in}} + \mathbf{s}_{\text{tb}}$ is trusted and converted into a peaked distribution; when they disagree strongly (high JSD), the distribution is blended toward uniform, spreading probability mass across all experts rather than committing to a potentially wrong one. The rule in Eq. (18) requires no additional learned parameters or tuned thresholds. The final Round-2 prediction is generated using $\boldsymbol{\alpha}$ and the same weight-level adapter merging as in Round-1 (Eq. (16)).

In summary, task-specific LoRA adapters prevent catastrophic forgetting through architectural isolation, since training on a new task updates only its dedicated adapter while all earlier adapters and the backbone remain frozen. Routing feature extraction projects frozen backbone representations through a fixed orthonormal matrix, keeping the routing space stable throughout the task sequence so GMM densities fitted at any point remain comparable as more tasks arrive. A per-task

diagonal GMM fitted by EM models each task's instruction distribution and provides log-posterior scores for task-agnostic routing, realized as hard argmax selection for the CodeTask benchmark or temperature-scaled soft adapter merging for the executable benchmark; for the latter, the iterative refinement pipeline recovers samples that exhaust all Round-1 candidates by combining instruction and traceback routing signals through the JSD-gated rule.

5. Evaluation

5.1. Research questions

The evaluation is organized around four research questions.

RQ1. How does GMM-Router perform relative to state-of-the-art continual learning baselines on a diverse, non-executable, eight-task code generation sequence?

RQ2. How does GMM-Router perform on a nine-language executable code generation benchmark judged by functional correctness, both on average and per programming language?

RQ3. To what extent does GMM-Router mitigate catastrophic forgetting and support knowledge transfer relative to the baselines, and how accurately does its GMM router assign inputs to the correct task-specific expert?

RQ4. Does the iterative refinement pipeline — traceback routing combined with JSD-gated score combination — improve performance on executable-benchmark samples that fail every Round-1 candidate?

5.2. Datasets

Both evaluation tracks introduced in Section 3 are used: the eight-task CodeTask benchmark (Table 1, Figure 3), scored with text-overlap metrics, addresses RQ1 and the routing-accuracy component of RQ3; the nine-language executable benchmark (Table 2), scored by unit-test execution, addresses RQ2 and RQ4, and its executable-track routing behavior additionally informs RQ3. Training proceeds through each benchmark's task sequence once, in the fixed order shown in Figure 3 for CodeTask and the language order Python, C++, Swift, Rust, C#, Java, PHP, TypeScript, Shell for the executable benchmark; validation after each task evaluates the model on every task seen so far, which is what makes the forgetting and transfer metrics defined in Section 5.3.3 (Eqs. (22)–(24)) computable, while the tables in Section 6 report the final scores after the full sequence has been learned. Maximum training size per task is capped at 100,000 examples, validation at 2,000, and test at 5,000, depending on the dataset (Table 1).

Table 3
Property comparison of baselines and GMM-Router. ✓ = has property; ✗ = lacks property. **Back.** = backbone frozen; **Isol.** = dedicated module per task; **Agn.** = no task identity needed at inference; **Repl.** = stores/replays raw data; **Rout.** = routes via learned feature distributions; **Shar.** = models cross-task parameter relationships; **Reg.** = applies parameter regularization; **Refine.** = iteratively refines failed predictions using execution feedback.

Method	Back.	Isol.	Agn.	Repl.	LoRA	Prompt	Rout.	Shar.	Reg.	Refine.
LoRA per task	✓	✓	✗	✗	✓	✗	✗	✗	✗	✗
EWC	✗	✗	✓	✗	✗	✗	✗	✗	✓	✗
L2P	✓	✗	✓	✗	✗	✓	✗	✗	✗	✗
ProgPrompt	✓	✓	✓	✗	✗	✓	✗	✗	✗	✗
SeqLoRA	✓	✗	✓	✗	✓	✗	✗	✗	✗	✗
O-LoRA	✓	✗	✓	✗	✓	✗	✗	✗	✓	✗
Online-CL-LLM	✓	✓	✓	✗	✓	✗	✓	✗	✗	✗
TreeLoRA	✓	✗	✓	✗	✓	✗	✗	✓	✗	✗
GMM-Router	✓	✓	✓	✗	✓	✗	✓	✗	✗	✓

5.3. Configurations

5.3.1. Base model and baselines

All experiments use Qwen2.5-Coder-1.5B (Hui et al., 2024), a decoder-only code language model (Section 2.2) that is large enough to provide strong code-generation ability while remaining practical for repeated continual learning runs. GMM-Router is compared against seven continual learning baselines chosen to cover the main methodological directions (Section 7), plus one task-identity upper bound.

LoRA per task trains an independent LoRA module per task with the backbone frozen and selects the module matching the true task label at inference (Hu et al., 2022); because no adapter is ever overwritten, it removes catastrophic forgetting by construction and serves as an optimistic upper bound that requires task identity and storage that grows linearly with K. *EWC* (Kirkpatrick et al., 2017) trains a single shared parameter set and penalizes changes to parameters the Fisher information marks as important for earlier tasks. *L2P* (Wang et al., 2022) keeps the backbone frozen and learns a shared pool of soft prompts retrieved per input. *ProgPrompt* (Razdaibiedina et al., 2023) learns a new soft prompt per task while freezing previously learned prompts. *SeqLoRA* trains one shared LoRA module sequentially across all tasks with no isolation mechanism, representing an aggressive, forgetting-prone form of parameter sharing. *O-LoRA* (Wang et al., 2023b) extends sequential LoRA with an orthogonality constraint between successive task updates. *Online-CL-LLM* (Cheng, Ye, Xu, Yan, Zhang, Liu, Huang and Feng, 2025) characterizes each PEFT block by a representative feature distribution and dynamically selects blocks per input without task labels, making it the baseline methodologically closest to GMM-Router. *TreeLoRA* (Qian et al., 2025) organizes adapters in a hierarchical tree by gradient similarity between tasks, sharing structure between related tasks while separating unrelated ones. Table 3 summarizes the baselines and GMM-Router along ten design properties.

Table 4
Hyperparameters of the instruction and traceback routers.

Hyperparameter	Instruction	Traceback
Routing layer ℓ^*	4	4
Routing dimension p	256	256
Max input length	2048	256
Truncation side	right	left
Variance floor $\sigma^2_{\min}$	0.02	10^{-3}
EM max iterations $T_{\max}$	50	100
EM tolerance δ	10^{-4}	10^{-4}
GMM components M	4	4
Training samples	up to 5,000	up to 600

5.3.2. Hyperparameters

For the CodeTask benchmark, the eight-task order follows Figure 3; for the executable benchmark, the nine-language order is Python, C++, Swift, Rust, C#, Java, PHP, TypeScript, Shell. All methods train for 3 epochs per task with batch size 32, learning rate 10^{-4}, warmup ratio 0.01, and weight decay 0.01. Padding tokens are masked in the labels (target value -100) so they do not contribute to the loss, and long inputs are truncated based on each dataset's input-length distribution.

LoRA-based methods use rank $r = 16$, `lora_alpha` $= 32$, `lora_dropout` $= 0.1$, applied to the attention query and value projections (`q_proj`, `v_proj`) with no trained bias parameters. After each adapter is trained, it is run on the held-out calibration split to generate $n = 5$ candidate predictions per problem (temperature 0.2, top-p $= 0.95$); the stored output — instruction, five candidates, pass/fail result, and error string per failed candidate — feeds both the traceback router and adapter refinement. Table 4 compares the instruction and traceback router hyperparameters; the traceback router's higher EM iteration budget compensates for its smaller, post-deduplication training set, and its input is truncated from the left so the final lines of a traceback — which carry the error type and message — are preserved. Adapter refinement continues training the existing per-task adapter at learning rate 5×10^{-5} for 3 epochs, weight decay 0.01, effective batch size 8, and a maximum prompt length of 4096 tokens to accommodate the concatenated instruction, buggy code, and traceback.

For Round-1 inference, the routing temperature is $\tau = 1.0$. On the CodeTask benchmark, hard routing is used with generation temperature 0.2, top-p $=$ 0.95, and up to 512 newly generated tokens. On the executable benchmark, soft routing is used, generating 5 candidates per sample (up to 2048 tokens) to support Pass@k evaluation. Round-2 inference applies only to the executable benchmark: per-router temperatures default to 1.0 and failed samples produce one new prediction with greedy decoding.

5.3.3. Evaluation metrics

The non-executable and executable tracks use different metrics because they capture different aspects of code generation quality. For the non-executable benchmark, BLEU (Papineni et al., 2002) measures n-gram overlap between a generated sequence and one or more references using modified precision p_n (each n-gram count in the hypothesis clipped by its maximum occurrence across references) and a brevity penalty BP that equals 1 when the hypothesis is at least as long as the reference and $\exp(1-r/c)$ otherwise, for hypothesis length c and reference length r:

$$\mathrm{BLEU} = \mathrm{BP} \cdot \exp\left(\sum_{n=1}^{4} \tfrac{1}{4} \log p_n\right). \tag{19}$$

CodeBLEU (Ren et al., 2020) extends BLEU with code-specific components, since code quality depends on syntactic and semantic structure as well as token overlap:

$$\begin{aligned}\mathrm{CodeBLEU} = \alpha \cdot \mathrm{NgramMatch} \\ + \beta \cdot \mathrm{WeightedNgramMatch} \\ + \gamma \cdot \mathrm{SyntaxMatch} \\ + \delta \cdot \mathrm{DataflowMatch},\end{aligned} \tag{20}$$

with $\alpha = \beta = \gamma = \delta = 0.25$. `NgramMatch` is standard BLEU-style precision; `WeightedNgramMatch` upweights language keywords and operators; `SyntaxMatch` compares abstract syntax trees by tree-edit distance; `DataflowMatch` compares variable-level data-flow relations, capturing semantic equivalence that token matching misses. Neither metric can fully distinguish code that is functionally correct from code that merely looks similar to a reference, which motivates the executable track.

For the executable benchmark, the main metric is Pass@k (Chen et al., 2021): the probability that at least one of $k \le n$ sampled candidates passes all unit tests, given n sampled candidates of which c pass,

$$\mathrm{Pass@}k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}, \tag{21}$$

defined as 1 when $n - c < k$. This paper reports Pass@1 (whether the first candidate is correct) and Pass@5 (whether any of five candidates is correct), which is informative in multilingual code generation where a problem may admit several correct implementations. Together, BLEU, CodeBLEU, and Pass@k let this paper evaluate both textual quality and functional correctness across the two complementary benchmark settings.

Three further metrics quantify continual learning behavior on top of either per-task metric: BLEU/-CodeBLEU on the CodeTask benchmark, or Pass@k on the executable benchmark. Let $a_{i,j}$ denote the

per-task metric value on task $\mathcal{T}_j$ after training on tasks $\mathcal{T}_1, \ldots, \mathcal{T}_i$. The Forgetting Measure (FM) (Wang, Zhang, Su and Zhu, 2023a) captures the drop from each task's best observed performance to its current performance:

$$f_{j,k} = \max_{i \in \{1,\ldots,k-1\}} (a_{i,j} - a_{k,j}), \qquad \mathrm{FM}_k = \frac{1}{k-1} \sum_{j=1}^{k-1} f_{j,k}. \tag{22}$$

Backward Transfer (BWT) measures how training on later tasks affects earlier ones:

$$\mathrm{BWT}_k = \frac{1}{k-1} \sum_{j=1}^{k-1} (a_{k,j} - a_{j,j}). \tag{23}$$

Forward Transfer (FWT) measures the extent to which learning previous tasks improves the model's performance on a new task before the model is trained on that task:

$$\mathrm{FWT}_k = \frac{1}{k-1} \sum_{j=2}^{k} \left(a_{j-1,j} - a_{0,j} \right), \tag{24}$$

where $a_{j-1,j}$ is the performance on task $\mathcal{T}_j$ after training on tasks $\mathcal{T}_1, \ldots, \mathcal{T}_{j-1}$ but before training on $\mathcal{T}_j$, and $a_{0,j}$ is the performance of the initial pretrained model on task $\mathcal{T}_j$ prior to any continual-learning training; $a_{j-1,j} - a_{0,j}$ thus quantifies the change in zero-shot performance on task $\mathcal{T}_j$ resulting from learning the preceding tasks. This definition follows the forward-transfer metric introduced by Lopez-Paz and Ranzato (2017), with the randomly initialized reference performance replaced by the initial pretrained-model performance, which provides a more meaningful baseline for pretrained code language models. Negative BWT indicates net forgetting; a positive FWT value indicates that knowledge acquired from previous tasks improves generalization to unseen tasks, a negative value indicates that previous training reduces the model's initial ability to perform the new task, and a value close to zero indicates that learning the preceding tasks has little effect on performance on future tasks.

6. Experimental Results

This section reports and discusses results for the four research questions in Section 5.1. LoRA per task serves as the upper-bound reference in every comparison because it uses the true task label at inference; every other method, including GMM-Router, must infer the task from the input alone.

6.1. RQ1: Performance on the CodeTask benchmark

Table 5 reports the final per-task and average scores on the CodeTask benchmark after the model has learned the full eight-task sequence.

Table 5
Final results (%) on the CodeTask benchmark (Qwen2.5-Coder-1.5B, $\tau = 2.0$). LoRA per task is the upper bound (task identity known at inference).

Method	CONCODE	CodeTrans	CodeSearchNet	BFP	KodCode	RunBugRun	TheVault	CoST	Avg.
LoRA per task	36.56	77.55	16.00	80.15	54.20	66.11	26.48	76.17	54.15
EWC	8.54	17.79	2.73	41.48	8.74	37.56	18.28	70.40	25.69
L2P	5.62	23.32	2.60	35.09	6.36	24.98	2.21	34.34	16.82
ProgPrompt	11.83	34.34	1.88	35.90	15.93	18.23	1.87	68.90	23.61
SeqLoRA	14.04	37.96	1.99	62.64	20.02	64.10	2.53	75.92	34.90
O-LoRA	10.84	36.09	13.80	77.24	41.77	65.89	21.21	75.34	42.77
Online-CL-LLM	22.52	47.97	14.65	68.57	25.22	64.83	15.24	71.87	41.36
TreeLoRA	14.85	34.58	2.12	44.24	17.27	78.18	2.16	83.20	34.58
GMM-Router	35.06	73.48	14.85	79.95	51.75	63.12	18.51	76.26	51.62

GMM-Router reaches the highest average (51.62%) among all task-agnostic methods, ahead of O-LoRA (42.77%) and Online-CL-LLM (41.36%) by a wide margin and close to the LoRA per task upper bound (54.15%). The advantage stems from three design choices acting together (Section 4): task-isolated LoRA adapters that prevent gradient interference, a stable routing space from the fixed orthonormal projection $\mathbf{P}$ applied to frozen backbone features, and hard routing that assigns each input deterministically to its best-matching expert via GMM log-posteriors under a uniform class prior.

GMM-Router is strongest on BFP (79.95%), CoST (76.26%), and CodeTrans (73.48%), all close to or matching the upper bound, indicating that dedicated experts with stable routing suffice for these structured tasks. It is weaker on TheVault (18.51%) and CodeSearchNet (14.85%), both code-to-text generation tasks whose broad, less distinctive input distributions are harder to separate by density estimation alone. Among the baselines, O-LoRA and Online-CL-LLM are strongest, reflecting the benefit of orthogonal subspaces and feature-based block selection, respectively; SeqLoRA (34.90%) and TreeLoRA (34.58%) trail, and EWC (25.69%), ProgPrompt (23.61%), and L2P (16.82%) are substantially weaker, indicating that shallow regularization or limited prompt capacity is insufficient for this diverse benchmark.

6.2. RQ2: Performance on the executable benchmark

Table 6 reports average Pass@1 and Pass@5 on the executable benchmark.

GMM-Router reaches 34.40% Pass@1 and 47.75% Pass@5. On Pass@1, SeqLoRA (37.51%) and TreeLoRA (36.84%) lead, benefiting from cross-task transfer that a shared or hierarchically shared adaptation space provides on this benchmark, which is homogeneous in task format across nine syntactically related languages. On Pass@5, GMM-Router leads all methods (47.75%), ahead of TreeLoRA (46.37%) and SeqLoRA (45.67%), indicating that soft routing produces a more diverse candidate set that collectively covers more of the problem space. GMM-Router's Pass@1 (34.40%) slightly exceeds the non-iterative LoRA per task upper bound (33.46%); this improvement is attributable to the iterative refinement pipeline recovering samples that fail

Table 6
Average Pass@1 and Pass@5 (%) on the executable benchmark. LoRA per task is the upper bound (task identity known, without iterative refinement).

Method	Avg. Pass@1 (%)	Avg. Pass@5 (%)
LoRA per task	33.46	42.19
EWC	0.09	0.22
L2P	8.03	11.75
ProgPrompt	28.01	40.00
SeqLoRA	37.51	45.67
O-LoRA	33.71	42.09
Online-CL-LLM	33.50	41.95
TreeLoRA	36.84	46.37
GMM-Router	34.40	47.75

Table 7
Per-language Pass@1 (%) on the executable benchmark ($\tau = 1.0$).

Method	Py	C++	Swift	Rust	C#	Java	PHP	TS	Shell
LoRA per task	43.59	6.80	59.20	22.64	1.20	18.87	36.40	93.20	19.20
EWC	0.80	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
L2P	7.60	2.40	24.80	1.51	7.60	0.00	10.80	17.60	0.00
ProgPrompt	31.60	16.80	57.20	4.15	3.60	13.96	30.80	78.80	15.20
SeqLoRA	45.52	13.60	62.00	10.57	24.00	22.26	33.20	98.40	28.00
O-LoRA	42.40	11.20	63.60	11.70	0.80	18.49	36.80	95.20	23.20
Online-CL-LLM	42.80	21.20	42.00	8.30	25.20	20.75	30.80	90.80	19.60
TreeLoRA	42.80	4.40	63.60	32.08	14.80	21.51	34.40	95.60	22.40
GMM-Router	38.40	5.60	66.40	19.62	2.80	23.40	34.80	92.80	25.80

every Round-1 candidate (RQ4, Section 6.4). GMM-Router also outperforms O-LoRA (33.71%/42.09%) and Online-CL-LLM (33.50%/41.95%), consistent with per-task GMM density estimation in a stable, fixed-projection routing space providing a more reliable task-distribution model than orthogonality constraints or feature-distribution matching alone. EWC (0.09% Pass@1) and L2P (8.03%) perform poorly, indicating that regularization and shallow prompt selection are insufficient for generating functionally correct multilingual code. The gap to SeqLoRA (−3.11pp Pass@1) reflects a trade-off examined further in Section 6.5.1: SeqLoRA's shared adaptation accumulates cross-language knowledge on this homogeneous benchmark but forgets severely on the diverse CodeTask sequence (34.90% vs. GMM-Router's 51.62%, Table 5).

6.2.1. Per-language analysis

Tables 7 and 8 report Pass@1 and Pass@5 for each programming language.

GMM-Router leads all methods on Swift (66.40% Pass@1, 80.00% Pass@5) and on Java Pass@5 (45.28%, well above the next best, SeqLoRA and O-LoRA at 28.30%), suggesting soft routing produces diverse candidates that cover more of the Java problem space. TypeScript is the easiest language overall (92.80% Pass@1, 100.00% Pass@5 for GMM-Router and most strong baselines). Rust shows a diversity benefit from soft routing: GMM-Router's Pass@1 (19.62%) trails TreeLoRA (32.08%), yet its Pass@5 (58.49%) exceeds

Table 8
Per-language Pass@5 (%) on the executable benchmark ($\tau = 1.0$).

Method	Py	C++	Swift	Rust	C#	Java	PHP	TS	Shell
LoRA per task	54.00	8.00	72.00	35.85	4.00	35.85	44.00	96.00	30.00
EWC	2.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
L2P	14.00	4.00	30.00	3.77	14.00	0.00	14.00	26.00	0.00
ProgPrompt	46.00	32.00	76.00	9.43	8.00	24.53	48.00	92.00	24.00
SeqLoRA	54.00	18.00	74.00	20.75	36.00	28.30	38.00	100.00	42.00
O-LoRA	56.00	14.00	72.00	24.53	4.00	28.30	46.00	100.00	34.00
Online-CL-LLM	50.00	30.00	54.00	11.32	32.00	30.19	40.00	98.00	32.00
TreeLoRA	54.00	8.00	70.00	50.94	24.00	26.42	44.00	100.00	40.00
GMM-Router	46.00	8.00	80.00	58.49	8.00	45.28	46.00	100.00	38.00

TreeLoRA's (50.94%). GMM-Router is weakest on C# (2.80%/8.00%) and C++ (5.60%/8.00%); for C#, methods with shared adaptation (SeqLoRA 24.00%, Online-CL-LLM 25.20%) perform substantially better, suggesting C# benefits from cross-language transfer that per-task isolation cannot exploit. Python (38.40% Pass@1) similarly falls behind SeqLoRA (45.52%), consistent with a Python distribution close enough to other languages in this benchmark that shared adaptation provides a useful inductive bias.

6.3. RQ3: Forgetting, transfer, and router assignment

6.3.1. Router assignment analysis

To understand which task distributions the GMM router can reliably separate, this subsection examines hard-routing confusion matrices ($\hat{k} = \text{argmax}_k\, \alpha_k(x)$) at the final training step. On CodeTask, hard routing is the actual inference method; on the executable benchmark, Round-1 uses soft routing, so this analysis instead exposes where the argmax decision fails and motivates the traceback router used in Round-2.

CodeTask benchmark. Figure 6 shows the row-normalized confusion matrix after the full eight-task sequence (percentage of 1,000 test samples per true task hard-routed to each expert). The average accuracy is 90.6%. BFP (98.3%) and CoST (99.5%) are routed almost perfectly because their distinctive input/output language combinations (Java bug patches; C++ source with C# target) produce compact, well-separated GMM regions. The two weakest tasks are TheVault (82.6%) and RunBugRun (86.4%). Two systematic confusion patterns explain most off-diagonal mass: CodeTrans draws 10.8% of TheVault and 5.8% of CONCODE samples because all three tasks share Java or C# as an input or output language; and CoST draws 9.0% of RunBugRun, 8.6% of CodeTrans, 6.7% of KodCode, and 6.6% of TheVault, reflecting both its C# output overlapping with CodeTrans/TheVault and code-to-code transformation tasks sharing structural embedding patterns with translation. These confusions align with the performance gaps in Table 5: TheVault and RunBugRun show the largest gaps to their per-task upper bounds.

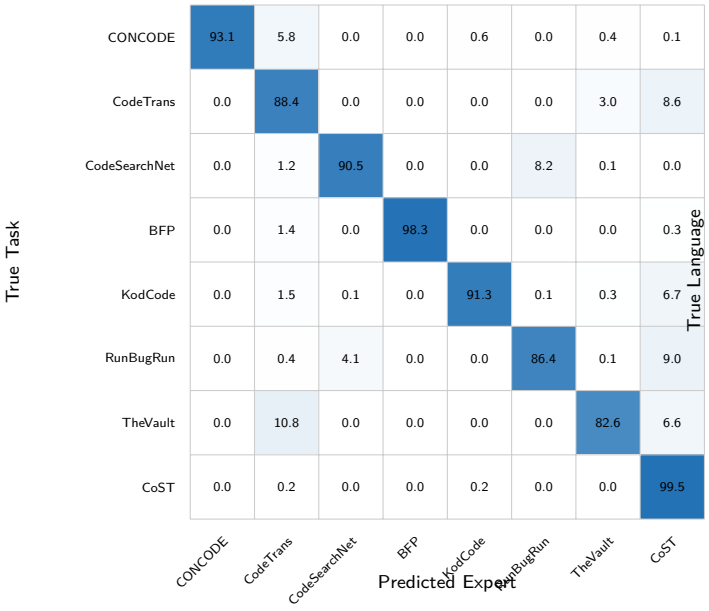


Figure 6: Hard-routing confusion matrix (%) on the CodeTask benchmark at the final training step. Rows = true task; columns = assigned expert. Diagonal entries indicate correct routing; darker cells indicate higher routing weight.

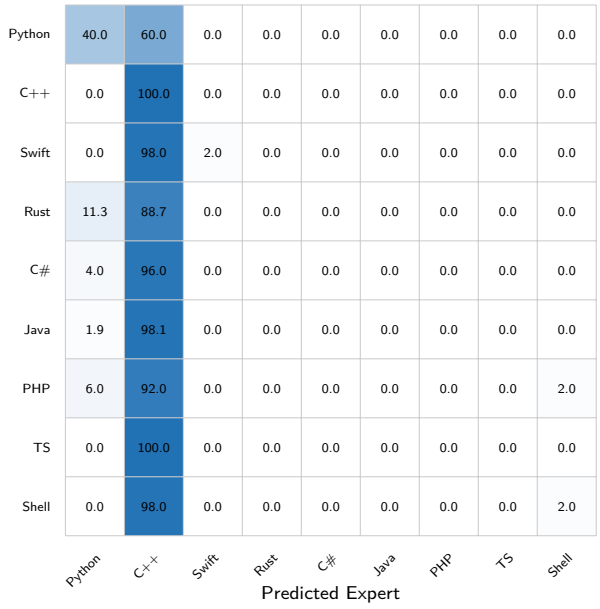


Figure 7: Hard-routing confusion matrix (%) of the *instruction* router on the executable benchmark at the final training step. The C++ column dominates, collecting most mass from every language.

Executable benchmark. Figure 7 shows the instruction router's confusion matrix after the full nine-language sequence. The average accuracy is only 15.8%, sharply lower than on CodeTask. C++ dominates as a single attractor, drawing between 60% (Python) and 100% (C++ itself and TypeScript) of every language's samples; Python is the only exception, correctly routing 40.0% of its own samples, while all other non-C++ languages score at most 2.0%. This collapse occurs because the nine languages share deep syntactic overlap — C-style braces, type keywords, similar control-flow constructs — and because C++, trained second in the sequence, established broad coverage of the routing feature space before later languages' GMMs could carve out well-separated regions.

Despite this, GMM-Router still reaches 34.40% Pass@1 (Table 6), because Round-1 uses soft, not hard, routing: the softmax at $\tau = 1.0$ distributes weight across every fitted expert, so even when the C++ GMM has the highest log-likelihood, other experts receive non-negligible weight, and the language expert that genuinely models the target can still dominate the merged generation. This effect is most visible for TypeScript, which has 0% hard-routing accuracy yet reaches 92.80% Pass@1 — close to the LoRA-per-task upper bound (93.20%) — indicating that soft mixing across experts approximates the dedicated TypeScript adapter closely.

Figure 8 shows the traceback router's confusion matrix, fitted on error tracebacks from failed Round-1 candidates rather than on instructions. The contrast is striking: average accuracy rises from 15.8% to 96.8%, with every language at or above 91.0%. This gap is explained by the information content of error messages: unlike instruction prompts, which share vocabulary across languages, error tracebacks carry language-specific identifiers (Python's `AttributeError`/`TypeError`; Java's `NullPointerException`; C++ segmentation faults and template-instantiation errors; Swift's distinctively annotated memory-access violations), letting the traceback GMM learn well-separated per-language regions where the instruction GMM cannot. The dominant off-diagonal direction is toward Shell (6.0–8.0% of Python and C++ samples), consistent with Shell capturing systems-level patterns — command execution, file-system access, subprocess and environment handling — that also occur in general-purpose languages.

6.3.2. Knowledge transfer and forgetting dynamics

This subsection evaluates GMM-Router, O-LoRA, SeqLoRA, and TreeLoRA — the four methods with complete per-step evaluation matrices — using BWT, FM, and FWT (Eqs. (22)–(24)).

CodeTask benchmark. Table 9 shows that GMM-Router achieves by far the lowest forgetting: BWT −1.01% and FM 1.01%, more than ten times smaller in magnitude than the next best, O-LoRA (BWT −11.36%, FM 11.41%); SeqLoRA (−22.10%) and TreeLoRA (−33.79%) forget increasingly severely. All four methods show large, broadly comparable FWT (+37.80% to +43.90%), indicating each learns each

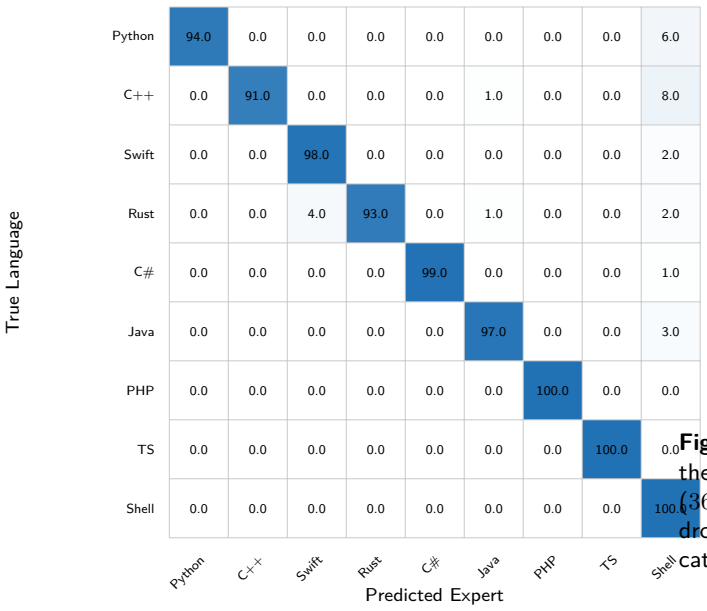


Figure 8: Hard-routing confusion matrix (%) of the *traceback* router on the executable benchmark at the final training step. The strong diagonal contrasts sharply with Figure 7.

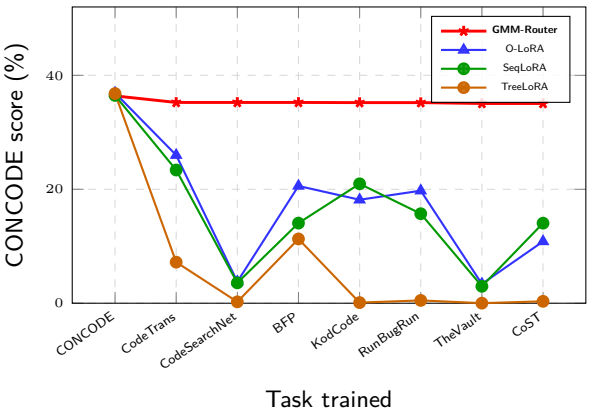


Figure 9: Retained CONCODE score after each task in the CodeTask sequence. GMM-Router remains nearly flat (36.39% → 35.06%); O-LoRA and SeqLoRA show severe drops with partial recovery; TreeLoRA suffers substantial catastrophic forgetting.

Table 9
BWT, FM, and FWT (%) on the CodeTask benchmark. Negative BWT indicates net forgetting; lower FM is better; higher FWT indicates stronger in-task learning.

Method	BWT (%) ↑	FM (%) ↓	FWT (%) ↑
GMM-Router	−1.01	1.01	+37.80
O-LoRA	−11.36	11.41	+37.97
SeqLoRA	−22.10	22.10	+39.79
TreeLoRA	−33.79	33.79	+43.90

new task well on first exposure. GMM-Router's low forgetting follows directly from per-task expert isolation: later training never modifies an earlier expert's LoRA parameters or its already-fitted GMM component. The only residual source of forgetting is routing drift, since adding new GMM components shifts the relative log-posterior of each expert, occasionally reducing the weight an earlier-task input receives on its own expert even though neither its adapter nor its GMM has changed. Figure 9 illustrates this: GMM-Router's retained CONCODE score declines only from 36.39% at its peak to 35.06% at the final step, while O-LoRA and SeqLoRA show sharp collapses with partial recovery, and TreeLoRA collapses from 36.80% to near zero after the first subsequent task and never recovers.

Executable benchmark. Table 10 shows a different pattern. GMM-Router and O-LoRA show negative BWT on both Pass@1 and Pass@5, indicating net forgetting, because the instruction router's GMM boundaries shift as later language experts are added, and this drift is more pronounced here than on CodeTask since all nine languages share deep syntactic overlap, so their GMM regions sit close together and a new language's GMM displaces boundaries more than in the type-diverse CodeTask sequence. SeqLoRA and TreeLoRA instead show positive BWT, benefiting directly from cross-language reinforcement in their shared parameters. On FM, GMM-Router is competitive on Pass@1 (1.77%, versus O-LoRA's 1.62%) and lowest on Pass@5 (3.83%): because each adapter is frozen after training, no single later step can catastrophically overwrite it, so forgetting accumulates gradually through routing drift rather than through parameter overwriting, keeping worst-case single-step drops small; SeqLoRA and TreeLoRA show the opposite trade-off, with positive BWT but much higher FM (6.92% and 6.38% Pass@1). All four methods show large FWT, and GMM-Router leads on Pass@5 FWT (+25.97%) because each language adapter trains exclusively on its own data without competing gradients from other languages in a shared parameter space, fitting the target distribution more tightly than shared-parameter methods must. Swift illustrates the isolation guarantee directly: GMM-Router's retained Swift Pass@1 starts and ends the remaining eight-task sequence at exactly 66.40% (dipping to 63.20% at one intermediate step before returning), whereas TreeLoRA erodes by 2.4pp and O-LoRA/SeqLoRA start lower but rise through cross-language reinforcement.

Taken together, RQ3 shows a clear cross-benchmark contrast: on the diverse CodeTask sequence, per-expert isolation reduces net forgetting by more than an order of magnitude relative to the best shared-parameter baseline; on the homogeneous executable benchmark, routing drift from closely packed language GMMs causes mild net forgetting, but GMM-Router still keeps FM competitive and leads on Pass@5 FWT.

Table 10
BWT, FM, and FWT (%) on the executable benchmark. Positive BWT indicates backward enhancement; lower FM is better; higher FWT indicates stronger in-task learning.

Method	BWT ↑		FM ↓		FWT ↑	
	P@1 (%)	P@5 (%)	P@1 (%)	P@5 (%)	P@1 (%)	P@5 (%)
GMM-Router	−1.22	−2.61	1.77	3.83	+20.94	+25.97
O-LoRA	−0.59	−1.36	1.62	4.51	+19.61	+18.85
SeqLoRA	+0.88	+0.28	6.92	10.47	+22.07	+20.99
TreeLoRA	+1.67	+3.68	6.38	8.44	+21.08	+18.63

Table 11
Ablation of Round-2 pipeline components on the executable benchmark. Best averages in bold.

Configuration	Avg. Pass@1 (%)	Avg. Pass@5 (%)
Round-1 only	33.04	41.66
Round-2, original adapter + JSD	**34.40**	**47.75**
Round-2, refined adapter + JSD	33.62	43.97

Table 12
Round-2 coverage and lift per language at the final training step. Languages sorted by lift, descending.

Language	Coverage (%)	Triggered/Total	Lift (%)
Rust	81.1	43/53	48.8
Java	67.9	36/53	19.4
Shell	64.0	32/50	3.1
C++	92.0	46/50	0.0
C#	92.0	46/50	0.0
Python	54.0	27/50	0.0
PHP	54.0	27/50	0.0
Swift	20.0	10/50	0.0
TypeScript	0.0	0/50	—

6.4. RQ4: Iterative refinement

GMM-Router's Round-2 pipeline (Section 4.5) triggers only for samples where all five Round-1 candidates fail. Table 11 ablates its two components: the traceback router with JSD-gated combination, and an optional adapter-refinement step.

Adding Round-2 with the original adapter and JSD-gated combination raises average Pass@1 by 1.36pp (33.04% → 34.40%) and Pass@5 by 6.09 points (41.66% → 47.75%), concentrated on Rust (+9.05% Pass@1, +39.62% Pass@5) and Java (+2.65% Pass@1, +13.18% Pass@5), the two languages where Round-1 candidates most often exhaust all five attempts. Table 12 breaks Round-2 coverage (fraction of samples triggering Round-2) and lift (fraction of triggered samples recovered) down by language at the final training step.

C++ and C# both trigger Round-2 on 92% of their test samples yet achieve zero lift: their failures stem from generation quality rather than routing mismatch, so redirecting to a different expert does not help. TypeScript never triggers Round-2 because every Round-1 candidate already passes, consistent with its near-perfect Table 7/8 scores. Refining each adapter on error-repair pairs (Table 11, third row) gives a mixed picture: Java Pass@5 rises further (to 47.17%), but Rust declines markedly (58.49% → 24.53% Pass@5), likely because repair targets are drawn from any passing candidate rather than one that specifically addresses the traceback shown, so the refined adapter can be trained on a repair signal that is not causally linked to the reported error (Section 6.5.3).

Table 13
Average Pass@1/Pass@5 (%) under four Round-2 routing strategies (original adapters).

Configuration	Avg. Pass@1 (%)	Avg. Pass@5 (%)
Round-1 only	33.04	41.66
Traceback alone	34.13	46.48
Unweighted sum	34.13	46.48
JSD-gated	**34.40**	**47.75**

Table 13 isolates the Round-2 routing rule itself, holding the original (non-refined) adapters fixed and comparing traceback-alone routing, an unweighted sum of instruction and traceback log-posteriors, and the JSD-gated combination of Eq. (18).

Traceback routing is the primary driver of Round-2 gains: moving from Round-1 to any Round-2 variant, Rust improves most (Pass@1 10.57% → 19.62%, Pass@5 18.87% → 58.49%), since Rust's distinctive borrow-checker and lifetime diagnostics let the traceback GMM identify and redirect these failures regardless of how the instruction router scored the problem; six of the nine languages show no change across Round-2 variants because their Round-1 candidates either already pass (TypeScript) or fail in ways rerouting cannot fix (C++, C#, Python, PHP, Swift). The unweighted sum produces results identical to traceback alone across all languages, because on samples that already exhausted five Round-1 attempts the traceback GMM assigns much sharper posterior mass to a single expert than the instruction GMM does, so summing the two log-posteriors does not change the top-ranked expert. JSD gating is what recovers the remaining conflicted samples: it behaves like the combined posterior when the two routers agree and blends toward uniform when they conflict, which prevents overconfident misrouting and lifts Java (Pass@5 35.85% → 45.28%, +9.43pp; Pass@1 +1.89pp) and Shell (Pass@5 +2pp) beyond what traceback-alone or the unweighted sum achieve; Rust is unaffected because the traceback router alone already accounts for most of its gain.

6.5. Discussion

6.5.1. Cross-benchmark comparison

Figure 10 plots each method's CodeTask average against its executable Pass@1, exposing a tension among the baselines: methods that maximize executable Pass@1 through parameter or gradient sharing

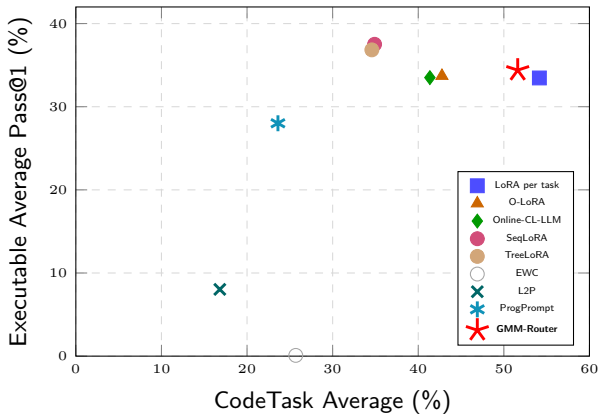


Figure 10: CodeTask average (%) vs. executable average Pass@1 (%) for each method. LoRA per task is the upper bound. Methods in the upper-right corner are preferred; methods that trade one benchmark for the other appear off-diagonal.

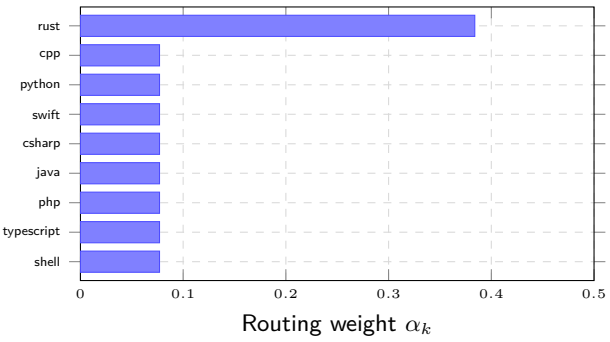


Figure 11: Round-2 routing weights α_k after traceback-GMM redistribution for the example in Section 6.5.2. The Rust expert rises from 0 to 0.384; the remaining weight spreads approximately evenly across the other eight experts.

(SeqLoRA, TreeLoRA) forget on the diverse CodeTask sequence, while methods that isolate tasks to protect CodeTask performance (O-LoRA, Online-CL-LLM) lack a dedicated task-agnostic routing mechanism and only reach moderate executable Pass@1. GMM-Router occupies a distinct position: 51.62% on CodeTask — 8.85pp above O-LoRA and 16.72pp above SeqLoRA — while its 34.40% executable Pass@1 exceeds the non-iterative upper bound, and its 47.75% Pass@5 leads all methods. This is consistent with per-task expert isolation and soft distribution-level routing being complementary: isolation prevents forgetting on diverse sequences, while soft routing and iterative refinement provide candidate diversity and recovery on homogeneous ones.

6.5.2. *Qualitative example: Round-2 recovery via traceback routing*

The following case illustrates why the traceback router recovers samples the instruction router cannot. For a Rust task (implementing `is_arithmetic_sequence`), the instruction router assigns $\alpha_{\text{cpp}} = 1.0$ with zero weight elsewhere, and all five Round-1 candidates fail to compile with `error[E0601]: ‘main‘ function not found`: the C++-trained adapter generates a bare function body following the C++ submission convention, where the harness supplies its own entry point, but Rust requires an explicit `fn main()` to compile as a standalone binary regardless of whether the function logic is correct. In Round-2, the traceback GMM redistributes routing weight to $\alpha_{\text{rust}} = 0.384$ (Figure 11), because `error[E0601]` is a diagnostic vocabulary specific to Rust's compilation model and absent from C++ output. The regenerated candidate is functionally identical to the Round-1 output except for the addition of `fn main()`, which the Rust adapter produces because its training data includes the standalone-binary scaffold. This example is representative of the pattern behind Table 12: the instruction GMM fails when generic problem phrasing makes two languages indistinguishable at the embedding level, while the traceback GMM succeeds because compilation errors are intrinsically language-specific diagnostics.

6.5.3. *Limitations and threats to validity*

Construct validity. BLEU and CodeBLEU, used on the CodeTask benchmark, cannot fully distinguish functionally correct code from code that merely resembles a reference (Section 5.3.3); this is mitigated by the executable benchmark's unit-test grading, but the two tracks are not directly comparable and are reported separately throughout.

Internal validity. All continual learning metrics (Eqs. (22)–(24)) depend on the fixed task order used during training (Section 5.3.2); a different order could change which tasks interfere with which, particularly on the executable benchmark where routing drift is order-dependent (Section 6.3.2). The repair pairs used for adapter refinement (Section 4.5) pair a failed candidate's traceback with any passing candidate rather than one verified to resolve that specific error, which is a plausible explanation for the Rust regression in Table 11 and is a limitation of the training signal rather than of the routing mechanism itself.

External validity. Experiments use a single 1.5B-parameter backbone (Section 5.3.1); routing behavior — particularly the C++ attractor effect in Figure 7 — may differ for backbones with different pretraining data or scale. The executable benchmark's test set is drawn from McEval and capped at 50–53 problems per language (Table 2), which bounds the precision of the per-language Pass@k estimates in Tables 7–8.

Conclusion validity. Per-task expert isolation, while effective at preventing forgetting, limits transfer between related tasks; this is most visible on C# and Rust in the executable benchmark (Section 6.2.1),

where shared-adaptation baselines benefit from cross-language transfer that GMM-Router cannot replicate without an explicit sharing mechanism. Routing quality also depends on how well the frozen backbone separates task distributions at the chosen layer ℓ^*; tasks with similar surface-level inputs but different output objectives, such as the nine syntactically related programming languages, are intrinsically harder to separate by density estimation alone (Section 6.3.1). Finally, adapter storage grows linearly with the number of tasks, which may become a practical constraint on much longer task sequences than the eight- and nine-task streams studied here.

7. Related Work

7.1. Continual learning methods

Regularization-based methods reduce forgetting by penalizing changes to parameters considered important for earlier tasks, without allocating separate modules per task. EWC (Kirkpatrick et al., 2017) estimates parameter importance from the Fisher information matrix and penalizes deviation from previous values; Learning without Forgetting (Li and Hoiem, 2017) instead preserves the model's output distribution via knowledge distillation; Synaptic Intelligence (Zenke et al., 2017) and Memory Aware Synapses (Aljundi et al., 2018) accumulate importance online. These methods are memory-efficient because they require no replay buffer, but because all tasks share one parameter space, their protection weakens as the number and diversity of tasks grows, which Table 5 confirms for EWC on the eight-task CodeTask sequence. GMM-Router sidesteps this failure mode by allocating a dedicated LoRA adapter per task (Section 4.1) rather than sharing one regularized parameter set.

Memory-based methods store information from earlier tasks and replay it during later training. GEM and A-GEM (Lopez-Paz and Ranzato, 2017; Chaudhry, Ranzato, Rohrbach and Elhoseiny, 2019) use stored examples to constrain gradients so earlier tasks are not harmed; DER++ (Buzzega, Boschini, Porrello, Abati and Calderara, 2020) replays stored logits to match past model behavior; Deep Generative Replay (Shin, Lee, Kim and Kim, 2017) generates pseudo-examples instead of storing real ones. These are strong baselines but increase per-step cost and conflict with settings, such as proprietary or privacy-sensitive code repositories, where retaining historical data is undesirable (Section 2.1). GMM-Router requires no stored training examples at inference time: its GMMs are fitted once from task-level routing embeddings during training and then frozen.

Architecture-based methods handle forgetting by partitioning model capacity across tasks. Progressive Neural Networks add a branch per task, PackNet reuses a fixed network through pruning, and HAT learns hard attention masks (Rusu, Rabinowitz, Desjardins, Soyer, Kirkpatrick, Kavukcuoglu, Pascanu and Hadsell, 2016; Mallya and Lazebnik, 2018; Serr'a, Suris, Miron and Karatzoglou, 2018); L2P (Wang et al., 2022) learns a shared, dynamically retrieved prompt pool, and Progressive Prompts (Razdaibiedina et al., 2023) keeps earlier prompts fixed while adding new ones per task. These are included as baselines in Section 5.3.1 (L2P, ProgPrompt); their limited representational capacity relative to full LoRA experts is consistent with their weaker CodeTask and executable-benchmark results (Tables 5–6).

LoRA-based continual learning is the branch most directly related to GMM-Router. A single LoRA trained sequentially (SeqLoRA in this paper) keeps storage flat but allows forgetting because all tasks share the same low-rank update (Hu et al., 2022); per-task LoRAs (the LoRA-per-task upper bound) prevent forgetting but require storage that grows linearly and either task identity or a router at inference. O-LoRA (Wang et al., 2023b) reduces interference by learning task updates in mutually orthogonal low-rank subspaces; GORP (Wang et al., 2025) uses low-rank gradient projection to expand the optimization space; TreeLoRA (Qian et al., 2025) organizes adapters hierarchically by gradient similarity. CoLoR (Wistuba, Sivaprasad, Balles and Zappella, 2023) studies whether LoRA itself is an effective continual learning mechanism for pretrained transformers; C-LoRA (Zhang, Bai, Yang and Liang, 2025) introduces a learnable routing matrix that manages low-rank updates across tasks while encouraging orthogonality; CL-LoRA (He, Duan and Zhu, 2025) combines task-shared and task-specific adapters with orthogonal structures and distillation for rehearsal-free continual learning. These methods reduce — but, because they still couple tasks through a partially shared subspace, do not eliminate — cross-task interference, and the number of usable orthogonal directions shrinks as more tasks are added (Section 1). GMM-Router instead removes coupling between adapters entirely by isolating each task in its own LoRA module and coordinating them only at inference, through routing rather than through a shared or orthogonalized subspace.

Online-CL-LLM (Cheng et al., 2025), evaluated as a baseline in Section 5.3.1, is the method closest in spirit to GMM-Router: it characterizes each PEFT block by a representative feature distribution and dynamically selects blocks per input without task labels, sharing GMM-Router's reliance on distributional information for task-agnostic routing. It differs in that it selects among existing PEFT blocks rather than combining them, whereas GMM-Router uses explicit per-task GMM posterior scores (Eq. (13)) to support both hard selection and soft, weight-level adapter merging (Eq. (16)), which Section 6.2 shows is responsible for

GMM-Router's Pass@5 lead on the executable benchmark.

7.2. Mixture of experts and density-based routing

The routing mechanism in Section 4 is related to the mixture-of-experts literature, in which expert networks specialize in different parts of the input space and a gating function determines their contribution. Jacobs et al. (1991) introduced adaptive mixtures of local experts with a learned gating network, and Jordan and Jacobs (1994) extended this to hierarchical mixtures trained with EM-style learning, establishing that routing and expert specialization admit a principled probabilistic interpretation. In modern deep learning, sparsely-gated mixture-of-experts layers (Shazeer et al., 2017) and Switch Transformers (Fedus et al., 2022) activate only a small number of experts per input using learned neural gating rather than an explicit density model, but the underlying motivation — routing different inputs to specialized modules — is the same. GMM-Router differs from this line of work in two respects relevant to the continual setting: its gate is an explicit, per-task Gaussian Mixture Model fitted independently for each task as it arrives (Section 4.3), rather than a single jointly trained gating network that would need to be retrained as new experts are added; and its routing features are extracted from a frozen backbone through a fixed orthonormal projection (Section 4.2) specifically so that GMM densities fitted at different points in the task sequence remain comparable, a stability requirement that does not arise in the single-training-run setting where standard MoE gating is typically deployed.

7.3. Code language models and evaluation

Pretrained code language models, reviewed in Section 2.2, provide the backbones on which all methods compared in this paper are built (Feng, Guo, Tang, Duan, Feng, Gong, Shou, Qin, Liu, Jiang and Zhou, 2020; Guo, Ren, Lu, Feng, Tang, Liu, Zhou, Duan, Svyatkovskiy, Fu et al., 2021; Guo, Lu, Duan, Wang, Zhou and Yin, 2022; Ahmad, Chakraborty, Ray and Chang, 2021; Wang et al., 2021, 2023c; Fried, Aghajanyan, Lin, Wang, Wallace, Shi, Zhong, Yih, Zettlemoyer and Lewis, 2023; Rozi'ere et al., 2023; Lozhkov et al., 2024; Hui et al., 2024). Continual learning for this domain was introduced by CodeTask-CL (Yadav et al., 2023), which this paper extends from four to eight tasks (Section 3.1) because a four-task sequence scored only by BLEU (Papineni et al., 2002) and CodeBLEU (Ren et al., 2020) under-represents both the length of realistic deployment sequences and the risk that text-similarity metrics reward superficially correct but functionally broken code. Execution-based evaluation, following the Codex/HumanEval methodology of Chen et al. (2021) and the multilingual MultiPL-E benchmark (Cassano et al., 2023), addresses the second gap directly but has not previously been combined with sequential, task-agnostic continual learning; the executable benchmark and deduplication pipeline in Section 3 are constructed specifically to close this gap.

8. Conclusion and Future Work

This paper studied task-agnostic, parameter-efficient continual learning for sequential code generation, a setting in which a model must adapt to a growing stream of generation, translation, summarization, refinement, and multilingual program-synthesis tasks without forgetting earlier ones and without access to task identity at inference time. We proposed GMM-Router, which trains a dedicated LoRA expert per task on a frozen backbone and routes each query using a diagonal Gaussian Mixture Model fitted on routing embeddings extracted through a fixed orthonormal projection, so that per-task density models remain comparable as the task sequence grows; for executable code generation, a second GMM fitted on runtime error tracebacks, combined with the instruction router through a Jensen–Shannon-divergence-gated rule, recovers samples that fail on every Round-1 attempt.

Across two complementary benchmarks — an eight-task extension of CodeTask-CL scored by text-overlap metrics and a nine-language executable benchmark scored by unit-test execution — the experiments in Section 6 answer the four research questions posed in Section 5.1. GMM-Router achieves the highest average score (51.62%) among all task-agnostic continual learning methods on CodeTask, approaching the task-identity upper bound (54.15%) with near-zero forgetting (BWT −1.01%, FM 1.01%; RQ1, RQ3). On the executable benchmark, it reaches 34.40% Pass@1 and the highest Pass@5 (47.75%) among all evaluated methods, exceeding the non-iterative upper bound on Pass@1 (RQ2). Router assignment analysis shows a sharp contrast between the two benchmarks: instruction-based hard routing reaches 90.6% accuracy on the type-diverse CodeTask sequence but only 15.8% on the syntactically homogeneous executable benchmark, where the complementary traceback router recovers 96.8% accuracy in Round-2 (RQ3). The iterative refinement pipeline contributes 1.36pp Pass@1 and 6.09 points Pass@5 over Round-1 alone, concentrated on Rust and Java, and JSD gating specifically recovers Java and Shell beyond what traceback routing alone achieves (RQ4). Taken together, these results indicate that per-task expert isolation and density-based probabilistic routing are complementary: isolation prevents forgetting on structurally diverse task sequences, and soft, distribution-level routing with iterative refinement provides candidate diversity and error recovery on structurally homogeneous ones, without requiring task labels, data replay, or a shared adapter subspace.

GMM-Router also has limitations, discussed in Section 6.5.3, that motivate three directions for future work. First, per-task expert isolation limits knowledge transfer between related tasks, most visibly on C# and Rust in the executable benchmark, where shared-adaptation baselines benefit from cross-language transfer that GMM-Router cannot currently exploit; a promising direction is a lightweight, jointly trained module shared beneath the task-specific adapter layers, or a distillation step between adapters for tasks whose GMMs the router already identifies as overlapping, turning the density model into a signal for both routing and selective knowledge consolidation. Second, adapter storage grows linearly with the number of tasks; dynamic expert merging — pruning or merging adapters whose GMMs exhibit high overlap in the routing space — or post-hoc adapter compression could keep GMM-Router practical for task sequences much longer than the eight- and nine-task streams studied here. Third, the repair pairs used for adapter refinement in Section 6.4 do not guarantee that the target solution resolves the specific error shown in the accompanying traceback, which is a plausible cause of the Rust regression observed when refinement is combined with JSD-gated routing (Table 11); constructing repair pairs with execution-guided search for patches that provably fix the reported error is a natural next step and may recover Rust's gains while preserving the improvement already observed on Java.

Acknowledgments

The author thanks Dr. Ngo Van Linh for supervision and feedback throughout this work, and Le Hai Nam for guidance on the direction and technical execution of the research. The author also thanks Nguyen Minh Tung and Cao Van Dong of the Foundation Models group at BKAI Laboratory for discussions and suggestions that improved this work, and Hanoi University of Science and Technology for the academic environment and computational resources that made this research possible.

Funding

TODO: insert funding source or a statement that no specific funding was received.

Declaration of competing interest

TODO: insert competing-interest declaration.

Data availability

TODO: insert links to the released CodeTask extension, the executable benchmark splits, and the code implementing GMM-Router, or a statement of the conditions under which they will be made available.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author used an AI coding assistant (Claude, Anthropic) to help restructure and condense prior thesis material into this manuscript's Journal of Systems and Software format. The author reviewed and edited the resulting material and takes full responsibility for the content of the publication.

References

Ahmad, W.U., Chakraborty, S., Ray, B., Chang, K.W., 2021. Unified pre-training for program understanding and generation, in: Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics, pp. 2655–2668.

Aljundi, R., Babiloni, F., Elhoseiny, M., Rohrbach, M., Tuytelaars, T., 2018. Memory aware synapses: Learning what (not) to forget, in: European Conference on Computer Vision, pp. 144–161.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al., 2021. Program synthesis with large language models. arXiv preprint arXiv:2108.07732 .

Bishop, C.M., 2006. Pattern Recognition and Machine Learning. Springer.

Broder, A.Z., 1997. On the resemblance and containment of documents, in: Proceedings of Compression and Complexity of Sequences, pp. 21–29.

Buzzega, P., Boschini, M., Porrello, A., Abati, D., Calderara, S., 2020. Dark experience for general continual learning: A strong, simple baseline, in: Advances in Neural Information Processing Systems, pp. 15920–15930.

Cassano, F., Gouwar, J., Nguyen, D., Nguyen, S., Phipps-Costin, L., Pinckney, D., Yee, M.H., Zi, Y., Anderson, C.J., Feldman, M.Q., et al., 2023. Multipl-e: A scalable and polyglot approach to benchmarking neural code generation. IEEE Transactions on Software Engineering 49, 3675–3691.

Chai, L., Liu, S., Yang, J., Yin, Y., Jin, K., Liu, J., Sun, T., Zhang, G., Ren, C., Guo, H., Wang, Z., Wang, B., Wu, X., Wang, B., Li, T., Yang, L., Duan, S., Li, Z., 2025. Mceval: Massively multilingual code evaluation, in: International Conference on Learning Representations.

Chaudhry, A., Ranzato, M., Rohrbach, M., Elhoseiny, M., 2019. Efficient lifelong learning with a-gem, in: International Conference on Learning Representations.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.d.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al., 2021. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 .

Cheng, X., Ye, J., Xu, H., Yan, M., Zhang, J., Liu, F., Huang, F., Feng, L., 2025. Exploiting presentative feature distributions for parameter-efficient continual learning of large language models, in: International Conference on Machine Learning.

De Lange, M., Aljundi, R., Masana, M., Parisot, S., Jia, X., Leonardis, A., Slabaugh, G., Tuytelaars, T., 2021. A continual learning survey: Defying forgetting in classification tasks. IEEE Transactions on Pattern Analysis and Machine Intelligence 44, 3366–3385.

Dempster, A.P., Laird, N.M., Rubin, D.B., 1977. Maximum likelihood from incomplete data via the em algorithm. Journal of the Royal Statistical Society: Series B 39, 1–38.

Fedus, W., Zoph, B., Shazeer, N., 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. Journal of Machine Learning Research 23, 1–39.

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., Zhou, M., 2020. Codebert: A pre-trained model for programming and natural languages, in: Findings of the Association for Computational Linguistics: EMNLP 2020, pp. 1536–1547.

Fried, D., Aghajanyan, A., Lin, J., Wang, S., Wallace, E., Shi, F., Zhong, R., Yih, W.t., Zettlemoyer, L., Lewis, M., 2023. Incoder: A generative model for code infilling and synthesis, in: International Conference on Learning Representations.

Goodfellow, I.J., Mirza, M., Xiao, D., Courville, A., Bengio, Y., 2013. An empirical investigation of catastrophic forgetting in gradient-based neural networks. arXiv preprint arXiv:1312.6211 .

Guo, D., Lu, S., Duan, N., Wang, Y., Zhou, M., Yin, J., 2022. Unixcoder: Unified cross-modal pre-training for code representation, in: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics, pp. 7212–7225.

Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., et al., 2021. Graphcodebert: Pre-training code representations with data flow, in: International Conference on Learning Representations.

He, J., Duan, Z., Zhu, F., 2025. Cl-lora: Continual low-rank adaptation for rehearsal-free class-incremental learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition.

Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., De Laroussilhe, Q., Gesmundo, A., Attariyan, M., Gelly, S., 2019. Parameter-efficient transfer learning for nlp, in: International Conference on Machine Learning, pp. 2790–2799.

Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., 2022. Lora: Low-rank adaptation of large language models, in: International Conference on Learning Representations.

Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., et al., 2024. Qwen2.5-coder technical report. arXiv preprint arXiv:2409.12186 .

Husain, H., Wu, H.H., Gazit, T., Allamanis, M., Brockschmidt, M., 2019. Codesearchnet challenge: Evaluating the state of semantic code search. arXiv preprint arXiv:1909.09436 .

Indyk, P., Motwani, R., 1998. Approximate nearest neighbors: Towards removing the curse of dimensionality, in: Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing, pp. 604–613.

Iyer, S., Konstas, I., Cheung, A., Zettlemoyer, L., 2018. Mapping language to code in programmatic context, in: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, pp. 1643–1652.

Jaccard, P., 1901. 'Etude comparative de la distribution florale dans une portion des alpes et du jura. Bulletin de la Soci'et'e Vaudoise des Sciences Naturelles 37, 547–579.

Jacobs, R.A., Jordan, M.I., Nowlan, S.J., Hinton, G.E., 1991. Adaptive mixtures of local experts. Neural Computation 3, 79–87.

Jordan, M.I., Jacobs, R.A., 1994. Hierarchical mixtures of experts and the em algorithm. Neural Computation 6, 181–214.

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A.A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., et al., 2017. Overcoming catastrophic forgetting in neural networks. Proceedings of the National Academy of Sciences 114, 3521–3526.

Kullback, S., Leibler, R.A., 1951. On information and sufficiency. The Annals of Mathematical Statistics 22, 79–86.

Lester, B., Al-Rfou, R., Constant, N., 2021. The power of scale for parameter-efficient prompt tuning, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pp. 3045–3059.

Li, Z., Hoiem, D., 2017. Learning without forgetting. IEEE Transactions on Pattern Analysis and Machine Intelligence 40, 2935–2947.

Lin, J., 1991. Divergence measures based on the shannon entropy. IEEE Transactions on Information Theory 37, 145–151.

Lopez-Paz, D., Ranzato, M., 2017. Gradient episodic memory for continual learning, in: Advances in Neural Information Processing Systems.

Lozhkov, A., Li, R., Ben Allal, L., Cassano, F., Lamy-Poirier, J., Tazi, N., Tang, A., Pykhtar, D., Liu, J., Wei, Y., et al., 2024. Starcoder 2 and the stack v2: The next generation. arXiv preprint arXiv:2402.19173 .

Lu, S., Guo, D., Ren, S., Huang, J., Svyatkovskiy, A., Blanco, A., Clement, C., Drain, D., Jiang, D., Tang, D., Li, G., Zhou, L., Shou, L., Zhou, L., Tufano, M., Gong, M., Zhou, M., Duan, N., Sundaresan, N., Deng, S.K., Fu, S., Liu, S., 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. arXiv preprint arXiv:2102.04664 .

Mallya, A., Lazebnik, S., 2018. Packnet: Adding multiple tasks to a single network by iterative pruning, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 7765–7773.

McCloskey, M., Cohen, N.J., 1989. Catastrophic interference in connectionist networks: The sequential learning problem, in: Psychology of Learning and Motivation. Academic Press. volume 24, pp. 109–165.

National Institute of Standards and Technology, 2015. Secure hash standard (shs), fips pub 180-4. Federal Information Processing Standards Publication 180-4.

Nguyen, D.M., Le, N.H., Dau, A.T.V., Nguyen, A.M., Nghiem, K., Guo, J., Bui, N.D.Q., 2023. The vault: A comprehensive multilingual dataset for advancing code understanding and generation, in: Findings of the Association for Computational Linguistics: EMNLP 2023, pp. 7988–8020.

Papineni, K., Roukos, S., Ward, T., Zhu, W.J., 2002. Bleu: A method for automatic evaluation of machine translation, in: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, pp. 311–318.

Parisi, G.I., Kemker, R., Part, J.L., Kanan, C., Wermter, S., 2019. Continual lifelong learning with neural networks: A review. Neural Networks 113, 54–71.

Prenner, J.A., Robbes, R., 2023. Runbugrun: An executable dataset for automated program repair. arXiv preprint arXiv:2304.01102 .

Qian, Y., Gao, J., Ding, N., Zhou, Y., Liang, J., Duan, Y., Liu, Z., Zhao, W.X., Tang, J., 2025. Treelora: Efficient continual learning via layer-wise loras guided by a hierarchical gradient-similarity tree, in: International Conference on Machine Learning.

Razdaibiedina, A., Mao, Y., Hou, R., Khabsa, M., Lewis, M., Almahairi, A., 2023. Progressive prompts: Continual learning for language models, in: International Conference on Learning Representations.

Ren, S., Guo, D., Lu, S., Zhou, L., Liu, S., Tang, D., Sundaresan, N., Zhou, M., Blanco, A., Ma, S., 2020. Codebleu: A method for automatic evaluation of code synthesis. arXiv preprint arXiv:2009.10297 .

Rozi'ere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X.E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., et al., 2023. Code llama: Open foundation models for code. arXiv preprint arXiv:2308.12950 .

Rusu, A.A., Rabinowitz, N.C., Desjardins, G., Soyer, H., Kirkpatrick, J., Kavukcuoglu, K., Pascanu, R., Hadsell, R., 2016. Progressive neural networks. arXiv preprint arXiv:1606.04671 .

Serr'a, J., Suris, D., Miron, M., Karatzoglou, A., 2018. Overcoming catastrophic forgetting with hard attention to the task, in:

International Conference on Machine Learning, pp. 4548–4557.
Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q.V., Hinton, G.E., Dean, J., 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, in: International Conference on Learning Representations.
Shin, H., Lee, J.K., Kim, J., Kim, J., 2017. Continual learning with deep generative replay, in: Advances in Neural Information Processing Systems.
Tufano, M., Watson, C., Bavota, G., Di Penta, M., White, M., Poshyvanyk, D., 2019. An empirical study on learning bug-fixing patches in the wild via neural machine translation. ACM Transactions on Software Engineering and Methodology 28, 19:1–19:29.
Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I., 2017. Attention is all you need, in: Advances in Neural Information Processing Systems.
Wang, C., Lyu, Y., Sun, Z., Jing, L., 2025. Continual gradient low-rank projection fine-tuning for llms, in: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics.
Wang, L., Zhang, X., Su, H., Zhu, J., 2023a. A comprehensive survey of continual learning: Theory, method and application. arXiv preprint arXiv:2302.00487 .
Wang, X., Chen, T., Ge, Q., Xia, H., Bao, R., Zheng, R., Zhang, Q., Gui, T., Huang, X., 2023b. Orthogonal subspace learning for language model continual learning. arXiv preprint arXiv:2310.14152 .
Wang, Y., Le, H., Gotmare, A.D., Bui, N.D.Q., Li, J., Hoi, S.C.H., 2023c. Codet5+: Open code large language models for code understanding and generation, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp. 1069–1088.
Wang, Y., Wang, W., Joty, S., Hoi, S.C.H., 2021. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, in: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pp. 8696–8708.
Wang, Z., Zhang, Z., Ebrahimi, S., Sun, R., Zhang, H., Lee, C.Y., Ren, X., Su, G., Perot, V., Dy, J., Pfister, T., 2022. Learning to prompt for continual learning, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 139–149.
Wei, Y., Wang, Z., Liu, J., Ding, Y., Zhang, L., 2023. Magicoder: Empowering code generation with oss-instruct. arXiv preprint arXiv:2312.02120 .
Wistuba, M., Sivaprasad, P.T., Balles, L., Zappella, G., 2023. Continual learning with low rank adaptation. arXiv preprint arXiv:2311.17601 Accepted at the Workshop on Distribution Shifts (DistShift), NeurIPS 2023.
Xu, Z., Liu, Y., Yin, Y., Zhou, M., Poovendran, R., 2025. Kodcode: A diverse, challenging, and verifiable synthetic dataset for coding. arXiv preprint arXiv:2503.02951 .
Yadav, P., Sun, Q., Ding, H., Li, X., Zhang, D., Tan, M., Ma, X., Bhatia, P., Nallapati, R., Ramanathan, M.K., Bansal, M., Xiang, B., 2023. Exploring continual learning for code generation models. arXiv preprint arXiv:2307.02435 .
Zenke, F., Poole, B., Ganguli, S., 2017. Continual learning through synaptic intelligence, in: International Conference on Machine Learning, pp. 3987–3995.
Zhang, X., Bai, L., Yang, X., Liang, J., 2025. C-lora: Continual low-rank adaptation for pre-trained models. arXiv preprint arXiv:2502.17920 .
Zhu, M., Suresh, T.R., Korotkova, E., Aghajanyan, A., O'Gorman, T., Iyyer, M., Reddy, S., 2022. Multilingual code snippets training for program translation, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 11783–11790.